

Type Theory with Opposite Types: A Paraconsistent Type Theory

Juan C. Agudelo-Agudelo¹ and Andrés Sicard-Ramírez²

¹Universidad de Antioquia, Medellín, Colombia. Email: juan.agudelo9@udea.edu.co

²Universidad EAFIT, Medellín, Colombia. Email: asr@eafit.edu.co

18th April 2021

Abstract

A version of intuitionistic type theory is extended with opposite types, allowing a different formalisation of negation and obtaining a *paraconsistent type theory* (PTT). The rules for opposite types in PTT are based on the rules of the so-called *constructible falsity*. A propositions-as-types correspondence between the many-sorted paraconsistent logic PL_S (a many-sorted extension of López-Escobar's *refutability calculus* presented in natural deduction format) and PTT is proven. Moreover, a translation of PTT into intuitionistic type theory is presented and some properties of PTT are discussed.

Keywords: Type theory, paraconsistent logic, constructible falsity, propositions-as-types correspondence.

Introduction

Intuitionistic type theory, also known as *constructive type theory* or *Martin-Löf type theory*, is a well-known formal system introduced by Martin-Löf in the 1970s as a proposal for the formalisation of intuitionistic mathematics. Through the so-called *propositions-as-types* (or *Curry-Howard*) *correspondence*, intuitionistic first-order logic (here abbreviated by IL) can be formalised into intuitionistic type theory.

From the point of view of mathematical constructivism, however, there are several objections against IL , and particularly against the intuitionistic definition of negation as $\sim A \stackrel{\text{def}}{=} A \supset \perp$, which can be interpreted as the impossibility of obtaining a construction for A . Heyting [8, p. 124], for instance, pointed out that

Serious objections against the use of negation in mathematics have been raised by Griss [...]. Though agreeing completely with Brouwer's basic ideas on the nature of mathematics, he contends that every mathematical notion has its origin in a mathematical construction, which can actually be carried out; if the construction is impossible, then the notion cannot be clear.

His objections lead Griss to propose a *negationless intuitionistic mathematics* (cf. [7]). Another text where objections against intuitionistic negation are mentioned is [23, p. 12], where van Dalen claims that

The justification [of the *ex falso* rule] in terms of constructions is not universally accepted, e.g. [Johansson, 1936] rejected the rule and formulated his so-called minimal logic, which has the same rules of intuitionistic logic with deletion of the *ex falso* rule.

Nelson also seemed to disagree with the intuitionistic definition of negation and provided in [19] a formalisation of arithmetic, named system $\mathbf{N}_1$, where Kleene’s realisability (called P-realisability in Nelson’s paper) is extended with a notion of N-realisability in a way that not only truth, but also falsity, are constructively defined in a symmetric way. Moreover, López-Escobar [12, Introduction] claims that

Intuitionistic negation has certain unpleasant characteristics. For example, if one accepts that there is no construction that proves an absurdity (as do most people) then a salient property of the construction π that proves ‘not- A ’ is that when π is applied to a particular non-existent construction (namely a proof of A) it yields another non-existent construction!

Taking into account these objections, López-Escobar [12] proposes a first-order logic defined by means of a sequent calculus, named *refutability calculus* (RFC), where rules for negation of compound formulae are defined by interpreting negation as refutation and by giving constructive rules for the notion of refutation. Furthermore, López-Escobar formalises arithmetic using RFC as the underlying logic and adding some number theoretic postulates; the resulting system is named PN. As it is pointed out in [12, p. 371], the two formalisations of arithmetic PN and $\mathbf{N}_1$ turn out to be very similar, the basic difference between them is that in the underlying logic of $\mathbf{N}_1$ the scheme $\neg A \supset (A \supset B)$ is a theorem, while in RFC it is not. This difference makes RFC be a paraconsistent logic (i.e. a logic supporting inconsistent but not trivial theories, which is formally defined in Section 1), while the underlying logic of $\mathbf{N}_1$ is not paraconsistent. The underlying logic of Nelson’s system $\mathbf{N}_1$ is formalised by Almukdad and Nelson [1] as the predicate calculus N. Although N is not paraconsistent, the authors also introduced in the same paper the paraconsistent first-order logic with constructible falsity $\mathbf{N}^-$. The logics RFC and $\mathbf{N}^-$ seems to be equivalent, however the authors do not know a proof of this equivalence.

Taking into consideration the objections that highlight the non-constructive nature of intuitionistic negation, in order to obtain a type theory where a negation with constructive properties can be formalised, it is reasonable to build a type system with one type constructor whose intended logical interpretation is the constructible falsity (and refutability) proposed by Nelson and López-Escobar, and that is what is done here by extending a version of intuitionistic type theory with what we shall call *opposite types*. The type theory obtained is called *paraconsistent type theory* (PTT), by reasons that will be clear below. Here, however, we only develop the idea of opposite type for function types, product types, disjoint union types, dependent product types and dependent sum types, leading open the extension of this idea for other types in future works. Because of that, here we shall take the $\rightarrow, \times, +, \Pi, \Sigma$ intensional fragment of intuitionistic type theory (as presented by Martin-Löf [17]), with two universes that will be described in Section 2, as our intuitionistic type theory of reference, and hereafter we shall refer to this version as ITT.

The version of PTT here presented is enough to formalising first-order logic and to prove a propositions-as-types correspondence. It is important to clarify that in PTT the negation will be

formalised using opposite types instead of the way in which it is done in intuitionistic type theory (where the negation of a type A , that can be interpreted as a proposition, is formalised as the function type from A to the empty type). Consequently, for our purposes here, we do not need an empty type in PTT.

Martin-Löf [18, p. 14] observes that

In Aristotelian logic, an affirmation was defined as a proposition in which something, called the predicate, is affirmed of something else, called the subject, and a denial was defined as a proposition in which the predicate is denied of the subject. Now, this is something that we have certainly abandoned in modern logic. Neither do we take categorical judgements to have subject-predicate form, nor do we treat affirmation and denial symmetrically.

Because Martin-Löf formalises negation in all his versions of intuitionistic type theory following the intuitionistic definition of negation, there is no symmetry between affirmations and denials in Martin-Löf’s type theories and all criticisms to the intuitionistic definition of negation mentioned above also apply to these type theories. Contrarily, if we understand denials as affirmations of negations—which is the usual way mathematicians understand denials since Frege (see, for instance, [18])—RFC and the underlying logic of N_1 allows us to recover the symmetry between affirmations and denials and provides a more adequate constructive formalisation of negation. Consequently, as in PTT we define opposite types based on the formalisation of negation in RFC and N_1 , allowing the formalisation of affirmations and denials in a constructive and symmetric way is a key feature of this type theory.

We are not the first to propose a type theory based on a logic with constructive negation. Wansing [26] introduced the typed lambda-calculus λ^C where types are the formulae of the propositional fragment of Nelson’s paraconsistent logic N^- (this fragment is also known as $N4$), and introduction and elimination rules are based on the rules of that logic. Moreover, a propositions-as-types correspondence and a semantics for λ^C is provided and it is pointed out that the assignment of types is not unique in this type system. However, important issues such as normalisation and strong normalisation for λ^C are left open. Later, Kamide [11] introduced the type system TA_{N4} that encodes Prawitz’s natural deduction system P_{N4} (a natural deduction system for $N4$). In TA_{N4} the terms used for the negation of implication, conjunction and disjunction are similar to the terms we use here for opposite of function types, products and disjoint unions, respectively. However, differently from [11], where the term constructors ι and ι^{-1} (called *type identity functions*) are included in the introduction and elimination rules for double negations, we here do not include any new kind of terms for the opposite of opposite types (which are the corresponding types of double negations in our system). We made that election because we want that terms of opposite types be the same that terms for non-opposite types, following the extension of the BHK-interpretation for RFC presented in [12]. In [11], strong normalisation for TA_{N4} is also not proven, but strong normalisation is proven for a subsystem of TA_{N4} without double negation rules. Finally, Kamide [10] introduced a different typed lambda-calculus for $N4$ for which he proved strong normalisation. In this new typed lambda-calculus, the assignment of types is also not unique and the term constructors neg and neg^{-1} are used in the double negation introduction and double negation elimination rules, respectively, in order to assure strong normalisation. Here, by the form in which the terms of opposite types are defined, the strong normalisation of PTT is a direct consequence of the strong normalisation of ITT.

Wansing [25] introduced the bi-connexive logic $2C$ and a typed lambda-calculus 2λ to encode deductions in this logic. In 2λ , terms of two sorts (with superscript $+$ for proofs and with super-

script – for dual proofs, or refutations) are used in order to assure uniqueness in the assignment of types and it is pointed out that the same idea can be used to assure unique typedness of terms in the type systems defined for N4. Although we show that some terms in PTT can have different types, we also indicate how the type theory can be modified in order to ensure the uniqueness of types (following similar ideas of such in [25]).

One important feature of intuitionistic type theory is that some types can be interpreted in several ways. For instance, the type $A \times B$ can be interpreted as the conjunction of propositions A and B , as the Cartesian product of the sets A and B or as the join of tasks A and B . The type system here defined also admits multiple interpretations for types, differently from the type systems mentioned in the previous two paragraphs where types are just formulae of some (propositional) logics and no other interpretations are left open. Moreover, PTT includes dependent types which is also an important difference with respect to the systems mentioned above where only non-dependent types are considered. The type theory proposed here aims to be a formal system adequate for formalising constructive mathematics with a constructive negation instead of the intuitionistic one. This is in contrast with the type theories mentioned in the previous paragraphs which are specifically designed only to encode deductions of the respective logical systems.

This article is structured as follows: In Section 1, a *first-order constructive paraconsistent logic* (PL) is defined by means of a natural deduction system. In PL, all logical constants, including negation, admit a constructive interpretation (exempt from the objections previously mentioned). Based on the rules for negation in PL, the system ITT is extended with opposite types in Section 2 obtaining the system PTT. In Section 3 it is presented a translation of PTT into ITT, which is used in Section 4 where it is proven a propositions-as-types correspondence between PTT and PL_S (a many-sorted extension of PL). The reason why the type theory obtained is a ‘paraconsistent type theory’ is explained in Section 5. The non-uniqueness of types in PTT, and how this possibly undesirable property can be corrected, is discussed in Section 6. In Section 7, some observations about equality in PTT are made. In Section 8, following some ideas of the so-called *Logics of formal inconsistency* (LFI’s) (see, for instance, Carnielli, Coniglio and Marcos [3]), it is explained how the ‘consistency’ of some types can be recovered in PTT. Finally, some concluding remarks and possibilities of future work are presented.

1 A first-order constructive paraconsistent logic

Nelson’s constructible falsity is formalised by Prawitz [20] by means of a natural deduction system. The *paraconsistent logic* PL that we shall consider here is a fragment of that system (to be more precise, PL is the formalisation of Nelson’s constructible falsity in [20, Appendix B] without the bottom particle and without the rule that connects intuitionistic negation and constructible falsity). The logical constants of PL are $\neg$, $\supset$, $\wedge$, $\vee$, $\forall$ and $\exists$, and their formulae are inductively defined as usual. Differently from IL, in PL there is not a bottom particle $\perp$ and the negation $\neg$ is a primitive logical constant that cannot be defined in terms of the other logical constants. Although in PL the logical constant $\vee$ can be defined in terms of $\neg$ and $\wedge$ by $A \vee B \stackrel{\text{def}}{=} \neg(\neg A \wedge \neg B)$, and the logical constant $\exists$ can be defined in terms of $\neg$ and $\forall$ by $(\exists x)A \stackrel{\text{def}}{=} \neg(\forall x)\neg A$, here we shall take $\vee$ and $\exists$ as primitive logical constants with the aim of making their behaviour more explicit.

In the natural deduction system for PL, the introduction and elimination rules for $\supset$, $\wedge$, $\vee$, $\forall$ and $\exists$ are just the same as in IL (as presented, for instance, in [20]). As negation represents falsity in PL, the negation of atomic formulae is treated in the same way as atomic formulae (both kind of

formulae represent atomic utterances, the first claiming the falsity of atomic propositions and the second claiming the truth of atomic propositions). Because of that, there are no introduction and elimination rules for the negation of atomic formulae but there are introduction and elimination rules for the negation of each one of the logical constants (including negation itself), such rules are presented in Table 1 (where given a formula A , a term t and a variable x , the expression $A(t/x)$ denotes the capture-avoiding substitution of the free occurrences of x in A by t).

$\frac{A}{\neg\neg A} \neg\neg\text{I}$	$\frac{\neg\neg A}{A} \neg\neg\text{E}$
$\frac{A \quad \neg B}{\neg(A \supset B)} \neg\supset\text{I}$	$\frac{\neg(A \supset B)}{A} \neg\supset\text{E}_1 \quad \frac{\neg(A \supset B)}{\neg B} \neg\supset\text{E}_2$
$\frac{\neg A}{\neg(A \wedge B)} \neg\wedge\text{I}_1$ $\frac{\neg B}{\neg(A \wedge B)} \neg\wedge\text{I}_2$	$\frac{\begin{array}{ccc} (\neg A) & & (\neg B) \\ & \vdots & \vdots \\ \neg(A \wedge B) & C & C \end{array}}{C} \neg\wedge\text{E}$
$\frac{\neg A \quad \neg B}{\neg(A \vee B)} \neg\vee\text{I}$	$\frac{\neg(A \vee B)}{\neg A} \neg\vee\text{E}_1 \quad \frac{\neg(A \vee B)}{\neg B} \neg\vee\text{E}_2$
$\frac{\neg A(t/x)}{\neg(\forall x)A} \neg\forall\text{I}$	$\frac{\begin{array}{ccc} (\neg A(y/x)) & & \\ & \vdots & \\ \neg(\forall x)A & C & \end{array}}{C} \neg\forall\text{E}$
$\frac{\neg A(y/x)}{\neg(\exists x)A} \neg\exists\text{I}$	$\frac{\neg(\exists x)A}{\neg A(t/x)} \neg\exists\text{E}$

Table 1: Introduction and elimination rules for negation in PL.

Negation rules for quantifiers are subject to the following conditions:

- Condition for $\neg\forall\text{I}$ and $\neg\exists\text{E}$: The term t must be free for x in A .
- Condition for $\neg\forall\text{E}$: The variable y may not occur free in $\neg(\forall x)A$, in C , or in any assumption on which C depends, other than $\neg A(y/x)$.
- Condition for $\neg\exists\text{I}$: The variable y must not occur free in any assumption on which $\neg A(y/x)$ depends nor in $\neg(\exists x)A$.

Considering the introduction and elimination rules of PL, derivability in this system is defined as usual in natural deduction systems. We shall use $\Delta \vdash_{\text{PL}} A$ to denote that the formula A is derivable from the set of formulae Δ in PL.

The following theorem states the equivalence between PL and RFC.¹ The languages of PL and RFC coincide, and the symbol $\vdash_{\text{RFC}}$ denotes the derivability relation in RFC.

Theorem 1.1. *If $\Delta \cup \{A\}$ is a set of formulae of PL, then $\Delta \vdash_{\text{PL}} A$ if and only if $\Delta \vdash_{\text{RFC}} A$.*

Proof. As RFC is a single succedent sequent calculus, this theorem can be proven following the same lines of the proof of Theorem 3.3.1 in [22]. $\square$

The equivalence between PL and RFC has as an immediate consequence that the Kripke semantics provided in [12] for RFC is also an adequate (that is, sound and complete) semantics for PL. This semantics allows us to prove (in Theorem 1.4) that PL is in fact a paraconsistent logic.²

Definition 1.2. Given a logic L with consequence relation $\Vdash_L$ and a negation $\neg$, a set of formulae Δ of L is [3]:

- *inconsistent* if there exists a formula A of L such that $\Delta \Vdash_L A$ and $\Delta \Vdash_L \neg A$,
- *trivial* if $\Delta \Vdash_L A$ for every formula A of L .

Definition 1.3. A logic L with consequence relation $\Vdash_L$ and a negation $\neg$ is *paraconsistent* if there exists a set of formulae Δ of L such that Δ is inconsistent but not trivial [3].

Paraconsistent logics differentiate between inconsistent and trivial theories, two concepts that are equivalent in IL (and also in classical logic and other logics).

Theorem 1.4. *PL is a paraconsistent logic.*

Proof. The schema $\neg A \supset (A \supset B)$ is not a theorem of RFC (cf. [12, p. 369]), therefore it is not a theorem of PL (by Theorem 1.1). Particularly, if we take A and B as being two different atomic formulae, we have that $\not\vdash_{\text{PL}} \neg A \supset (A \supset B)$ (which can be proven by using the semantics for RFC in [12]). Consequently, taking $\Delta = \{A, \neg A\}$, we have that Δ is an inconsistent but not trivial theory (otherwise, we would have that $A, \neg A \vdash_{\text{PL}} B$, and by the deduction theorem, which is valid for PL, it would follow that $\vdash_{\text{PL}} \neg A \supset (A \supset B)$). $\square$

Theorem 1.1 also allows us to extend the well-known BHK-interpretation of IL to PL, just as it is made in [12] for RFC. By taking the notion of ‘refutation’ as a primitive one (so as it is the notion of ‘proof’) and considering that a construction c proves $\neg A$ iff c refutes A , the refutation for each one of the logical constants in PL is interpreted in the following way:

- c refutes $\neg A$ if and only if c proves A .
- c refutes $A \wedge B$ if and only if $c = (i, d)$ with i either 0 or 1, and if $i = 0$, then d refutes A , and if $i = 1$, then d refutes B .
- c refutes $A \vee B$ if and only if $c = (d, e)$ and d refutes A and e refutes B .

¹Although the authors believe that PL and RFC are equivalent to the paraconsistent logic N^- introduced by Almkudad and Nelson [1], this is not a question to be addressed here. We simply shall take PL as our paraconsistent logical system of reference and its equivalence with RFC is sufficient for our purposes.

²Although RFC is a paraconsistent logic, and it is related with the failure of $\neg A \supset (A \supset B)$ in such logic, the paraconsistency of RFC is not explicitly stated in [12].

- c refutes $A \supset B$ if and only if $c = (d, e)$ and d proves A and e refutes B .
- c refutes $(\forall x)A$ if and only if $c = (a, d)$ and d refutes $A(a/x)$.
- c refutes $(\exists x)A$ if and only if c is a general method of construction such that given any individual (i.e. construction) a from the domain of objects under consideration, $c(a)$ (i.e. c applied to a) refutes $A(a/x)$.

The main idea of López-Escobar's extension of the BHK-interpretation is that not only proofs can be viewed as constructions (that testifies the truth of propositions), but that also refutations can be viewed as constructions (that testifies the falsity of propositions), and negation represents falsity. According to this idea, a proof of a negated formula $\neg A$ is the same that a refutation of A , then the negation of an atomic formula behaves like an atomic formula, and the notion of refutation is constructively defined for each one of the logical constants by the rules in Table 1. For instance, as a prime number can be defined as a natural number greater than 1 whose unique divisors are 1 and himself, which can be expressed as $x > 1 \wedge (\forall y)(y \mid x \supset (y = 1 \vee y = x))$, a proof of the negation (or a refutation) of the proposition '8 is a prime' is a pair $(1, (a, b))$, where a is 2 (or 4) and b is a proof of ' $2 \mid 8 \wedge (2 \neq 1 \wedge 2 \neq 8)$ ' (or of ' $4 \mid 8 \wedge (4 \neq 1 \wedge 4 \neq 8)$ ').³

Note that there is no need of supposing that there exists a construction that proves '8 is prime' and derive from this a construction that proof $\perp$, as it is the case in IL. We think that López-Escobar's interpretation of negation as falsity, and view of refutations as constructions that serve as witnesses to falsity, is more adequate for the formalisation of constructive mathematics than the intuitionistic interpretation of negation, because it states how constructions for negated formulae can be effectively build, while in the intuitionistic view the 'construction' for the negation of a formula A can only be understood as a procedure that shows the impossibility of obtaining a construction for A . Moreover, the example above shows that, at least in some cases, López-Escobar's formalisation of negation captures a mathematician's reasoning for constructively proving a negation in a more natural way.

In the rest of this section, we shall adapt some properties of the logic 2Int (an extension of N4 introduced by Wansing [24]) to PL. We shall use symbol $\equiv$ to denote equivalence in PL; that is, $A \equiv B$ will denote $(A \supset B) \wedge (B \supset A)$. An important feature of PL is that in this logic the substitution by equivalents fails. For instance, for different atomic formulae A and B , we have that $\vdash_{\text{PL}} \neg(A \rightarrow B) \equiv (A \wedge \neg B)$ and $\not\vdash_{\text{PL}} \neg\neg(A \rightarrow B) \equiv \neg(A \wedge \neg B)$ (which can be proven by using the semantics for RFC in [12]). However, a 'strong equivalence' can be defined in PL as follows.

Definition 1.5. Two formulae A and B of PL are *strongly equivalent*, which we shall denote by $A \equiv_s B$, if not only A and B are equivalent but also $\neg A$ and $\neg B$ are equivalent (i.e. $A \equiv_s B \stackrel{\text{def}}{=} (A \equiv B) \wedge (\neg A \equiv \neg B)$).

Substitution by equivalents is valid for strong equivalence as stated by the following theorem.

Theorem 1.6. *Let A, B and C be formulae of PL. If $\vdash_{\text{PL}} A \equiv_s B$ and C' is the result of substituting some occurrences of A by B in C , then $\vdash_{\text{PL}} C \equiv_s C'$.*

Proof. By structural induction on the formula C . □

Another interesting property of PL, which will be useful in Section 3, is that negations can be carried over to atomic formulae as explained below.

³As illustrated in the example, similarly to what happens in the demonstration of existential propositions, the witness to prove the falsity of universal propositions may not be unique.

Definition 1.7. A formula A of PL is in *negative normal form* if negations in A are applied only to atomic formulae.

Notation. Hereafter, the symbol $\diamond$ will be used as a placeholder for the logical binary constants $\supset$, $\wedge$ and $\vee$; and the symbol $\#$ will be used as a placeholder for the logical constants $\forall$ and $\exists$.

Definition 1.8. Given a formula A of PL, its *negative normal form*, denoted $\text{NNF}(A)$, is recursively defined by:

$$\begin{aligned} \text{NNF}(P(t_1, \dots, t_n)) &= P(t_1, \dots, t_n), & (P \text{ is an } n\text{-ary predicate}) \\ \text{NNF}(\neg P(t_1, \dots, t_n)) &= \neg P(t_1, \dots, t_n), \\ \text{NNF}(A \diamond B) &= \text{NNF}(A) \diamond \text{NNF}(B), \\ \text{NNF}(\#x.A) &= \#x.\text{NNF}(A), \\ \text{NNF}(\neg\neg A) &= \text{NNF}(A), \\ \text{NNF}(\neg(A \supset B)) &= \text{NNF}(A) \wedge \text{NNF}(\neg B), \\ \text{NNF}(\neg(A \wedge B)) &= \text{NNF}(\neg A) \vee \text{NNF}(\neg B), \\ \text{NNF}(\neg(A \vee B)) &= \text{NNF}(\neg A) \wedge \text{NNF}(\neg B), \\ \text{NNF}(\neg(\forall x)A) &= (\exists x)\text{NNF}(\neg A), \\ \text{NNF}(\neg(\exists x)A) &= (\forall x)\text{NNF}(\neg A). \end{aligned}$$

Theorem 1.9. $\vdash_{\text{PL}} A \equiv \text{NNF}(A)$, for every formula A of PL.

Proof. By structural induction on the formula A (taking care of not using substitution by equivalents, because this property fails in PL as mentioned above). In the base step, A must be an atomic formula or the negation of an atomic formula. $\square$

Although in PL every formula is equivalent with its negative normal form, some formulae are not strongly equivalent with their negative normal form. For instance, $\not\vdash_{\text{PL}} \neg(P \supset Q) \equiv_s \text{NNF}(\neg(P \supset Q))$ (where P and Q are different atomic formulae), which can be proven using the semantics for RFC.

2 Type theory with opposite types

Based on the formalisation of negation in PL, we propose here a *paraconsistent type theory* (PTT). The theory PTT is obtained by adding to ITT what we shall called *opposite types*. As we mentioned in the introduction, here we only develop the idea of opposite type for function types, product types, disjoint union types, dependent product types and dependent sum types, leaving open the extension of this idea for other kind of types in future works. Although the notion of opposite type is only partially developed here, the idea is that, for each type A , there exists in PTT an opposite type denoted $\overline{A}$. The formation rule for opposite types is then⁴

$$\frac{A : \text{Set}}{\overline{A} : \text{Set}}.$$

Opposites of basic types behave like basic types. Then, introduction, elimination and equality (or computation) rules of opposite types must be defined for each one of the type constructors

⁴We use here ‘Set’ instead of ‘Type’, as it is usual in intuitionistic type theory.

(including itself). The introduction and elimination rules for opposite types are presented in Table 2. By reasons of space and taking into consideration that the terms used in the introduction and elimination rules of opposite types already exist in ITT, the equality rules are defined just in the same way as in ITT and we omit their presentation here.

$\frac{a : A}{a : \overline{\overline{A}}}$	$\frac{a : \overline{\overline{A}}}{a : A}$
$\frac{a : A \quad b : \overline{B}}{(a, b) : \overline{A \rightarrow B}}$	$\frac{c : \overline{A \rightarrow B}}{p(c) : A} \quad \frac{c : \overline{A \rightarrow B}}{q(c) : \overline{B}}$
$\frac{a : \overline{A}}{i(a) : \overline{A \times B}}$ $\frac{b : \overline{B}}{j(b) : \overline{A \times B}}$	$\frac{(x : \overline{A}) \quad (y : \overline{B}) \quad \vdots \quad c : \overline{A \times B} \quad d(x) : C(i(x)) \quad e(y) : C(j(y))}{D(c, (x)d(x), (y)e(y)) : C(c)}$
$\frac{a : \overline{A} \quad b : \overline{B}}{(a, b) : \overline{A + B}}$	$\frac{c : \overline{A + B}}{p(c) : \overline{A}} \quad \frac{c : \overline{A + B}}{q(c) : \overline{B}}$
$\frac{a : A \quad b : \overline{B(a)}}{(a, b) : \overline{(\Pi x : A)B(x)}}$	$\frac{(x : A, y : \overline{B(x)}) \quad \vdots \quad c : \overline{(\Pi x : A)B(x)} \quad d(x, y) : C((x, y))}{E(c, (x, y)d(x, y)) : C(c)}$
$\frac{(x : A) \quad \vdots \quad b(x) : \overline{B(x)}}{(\lambda x)b(x) : \overline{(\Sigma x : A)B(x)}}$	$\frac{b : \overline{(\Sigma x : A)B(x)} \quad a : A}{Ap(b, a) : \overline{B(a)}}$

Table 2: Introduction and elimination rules for opposite types in PTT.

Contexts in PTT are defined in the same way than in ITT, and we shall use $\Gamma \vdash_{\text{PTT}} a : A$ to denote that $a : A$ is derivable from context Γ in PTT.

Definition 2.1. Let A and B two types of PTT. We shall say that A and B are *equivalent*, which will be denoted by $A \equiv B$, if a term t is of type A if and only if t is of type B , and we shall say that A and B are *strongly equivalent*, which will be denoted by $A \equiv_s B$, if not only A and B are equivalent but also $\overline{A}$ and $\overline{B}$ are equivalent.

Analogously as $\vee$ and $\exists$ can be defined in PL by means of the other logical constants, the type constructors $+$ and Σ , which are not definable in ITT from other types, can be defined in PTT by $A + B \stackrel{\text{def}}{=} \overline{\overline{A} \times \overline{B}}$ and $(\Sigma x : A)B \stackrel{\text{def}}{=} \overline{(\Pi x : A)\overline{B}}$ (because $A + B \equiv_s \overline{\overline{A} \times \overline{B}}$ and $(\Sigma x : A)B \equiv_s \overline{(\Pi x : A)\overline{B}}$).

Moreover, as in ITT, the type constructor $\rightarrow$ can be defined in PTT by $A \rightarrow B \stackrel{\text{def}}{=} \Pi x : A. B$, when x is not free in B (because $A \rightarrow B \equiv_s \Pi x : A. B$, when x is not free in B). However, while the type constructor $\times$ can be defined in ITT by means of Σ (namely $A \times B \stackrel{\text{def}}{=} (\Sigma x : A) B$, when x is not free in B), this definition is not valid in PTT because $\overline{A \times B} \not\equiv (\Sigma x : A) \overline{B}$ (canonical elements of $\overline{A \times B}$ are injections while canonical elements of $(\Sigma x : A) \overline{B}$ are λ -abstractions). Consequently, we could consider that PTT have only $\times, \Pi$ and opposite types, but with the aim of making the behaviour of each one of the constructors $\rightarrow, \times, +, \Pi, \Sigma$ in PTT more clear we shall consider all of them as primitive.

In order to prove a propositions-as-types correspondence for PTT (in Section 4), we assume that PTT and also ITT have two universes, U_0 (closed under $\rightarrow, \times, +, \Pi, \Sigma$ in both type systems and also closed under opposite types in PTT) and U_1 (closed only under $\rightarrow$ in both type systems), with $U_0 : U_1$. Universe U_1 is necessary to allow the formalisation of first-order predicates and is closed only under function types in order to facilitate the proof of the propositions-as-types correspondence.

Following usual terminology, the elements of U_0 and U_1 will be called *small types* and *large types*, respectively. If $x : A$ is a declaration in a context, the variable x will be called a *term variable* if A is a small type or a *type variable* if A is a large type. In the propositions-as-types correspondence, term variables will be used to represent the variables and function symbols of the logic and will also be used as proof terms of assumptions of logical theories, while type variables will be used to represent the sorts and predicate symbols of the logic.

Note that some type variables should generate new small types by application, and by the closure conditions of U_0 in PTT their opposite types will also be small types. For instance, for the context $\Gamma = \langle A : U_0, B : U_0, a : A, b : B, P : A \rightarrow B \rightarrow U_0 \rangle$, we have that $\Gamma \vdash_{\text{PTT}} \text{Ap}(\text{Ap}(P, a), b) : U_0$ and consequently $\Gamma \vdash_{\text{PTT}} \overline{\text{Ap}(\text{Ap}(P, a), b)} : U_0$. Under the propositions-as-types correspondence, the type $\text{Ap}(\text{Ap}(P, a), b)$ will correspond to the atomic predicate $P(a, b)$, while the type $\overline{\text{Ap}(\text{Ap}(P, a), b)}$ will correspond to the negation of $P(a, b)$.

It is important to highlight here that our choice of terms for opposite types has the advantage that strong normalisation for PTT is a direct consequence of the strong normalisation for ITT,⁵ because no new term constructors or reduction rules are added to the system.

3 A translation of PTT into ITT

In an analogous way as any formula of PL can be converted into an equivalent formula in negative normal form, any type of PTT can be converted into an ‘equivalent type’ in ‘opposite normal form’ (where the opposite type constructor is applied only to basic types). Here we do not defined this conversion, but we use the idea to define a translation of PTT into ITT. Such translation is used in the proof of a propositions-as-types correspondence for PTT (in Section 4). In the translation, the meta-variable X ranges over term and type variables and the meta-variable P ranges only over type variables.

Definition 3.1. The *translation* H of PTT into ITT is inductively defined.

$$\begin{aligned} H(U_0) &= U_0, & H(U_1) &= U_1, \\ H(A \rightarrow B) &= H(A) \rightarrow H(B), & H(A \times B) &= H(A) \times H(B), \end{aligned}$$

⁵Strong normalisation for a system that extends ITT is proven by Martin-Löf [15], which implies strong normalisation for ITT.

$$\begin{aligned} H(A + B) &= H(A) + H(B), & H((\Pi x : A)B) &= (\Pi x : H(A))H(B), \\ H((\Sigma x : A)B) &= (\Sigma x : H(A))H(B), \end{aligned}$$

$$\begin{aligned} H(\overline{P}) &= P^o, & H(\overline{\mathbf{Ap}(M, N)}) &= \mathbf{Ap}(H(\overline{M}), H(\overline{N})), \\ H(\overline{A \rightarrow B}) &= H(\overline{A}) \times H(\overline{B}), & H(\overline{A \times B}) &= H(\overline{A}) + H(\overline{B}), \\ H(\overline{A + B}) &= H(\overline{A}) \times H(\overline{B}), & H(\overline{(\Pi x : A)B}) &= (\Sigma x : H(A))H(\overline{B}), \\ H(\overline{(\Sigma x : A)B}) &= (\Pi x : H(A))H(\overline{B}), \end{aligned}$$

$$\begin{aligned} H(X) &= X, & H((\lambda x)M) &= (\lambda x)H(M), \\ H(\mathbf{Ap}(M, N)) &= \mathbf{Ap}(H(M), H(N)), & H((M, N)) &= (H(M), H(N)), \\ H(\mathbf{p}(M)) &= \mathbf{p}(H(M)), & H(\mathbf{q}(M)) &= \mathbf{q}(H(M)), \\ H(\mathbf{i}(M)) &= \mathbf{i}(H(M)), & H(\mathbf{j}(M)) &= \mathbf{j}(H(M)), \end{aligned}$$

$$\begin{aligned} H(\mathbf{D}(M, (x)N(x), (y)L(y))) &= \mathbf{D}(H(M), (x)H(N)(x), (y)H(L)(y)), \\ H(\mathbf{E}(M, (x, y)N(x, y))) &= \mathbf{E}(H(M), (x, y)H(N)(x, y)), \end{aligned}$$

$$H(\langle x_1 : A_1, \dots, x_n : A_n \rangle) = \langle H(x_1 : A_1), \dots, H(x_n : A_n) \rangle,$$

where $H(x_i : A_i)$ is $x_i : H(A_i)$ if A_i is a small type or $x_i : H(A_i), x_i^o : H(A_i)$ if A_i is a large type.

Note that we are considering that variables in contexts of **PTT** are not overlined. Thus, overlined expressions in **PTT** always correspond to opposite types, while variables with superscript o in **ITT** are just variables with convenient names to represent the opposite of basic types declared in contexts.

Theorem 3.2. *If $\Gamma \vdash_{\mathbf{PTT}} a : A$, then $H(\Gamma) \vdash_{\mathbf{ITT}} H(a) : H(A)$.*

Proof. By induction on the derivation in **PTT**. □

4 Propositions-as-types correspondence for **PTT**

Barendregt [2] proved a propositions-as-types correspondence between **PRED** (minimal intuitionistic first-order logic)⁶ and $\lambda\mathbf{P}$ (a type system in the so-called λ -cube). Following the same lines of such proof, we shall prove in this section a propositions-as-types correspondence between a many-sorted extension of **PL** and **PTT**. The sketch of the proof is the following:

1. **PL** is extended with sorts, obtaining the many-sorted logic $\mathbf{PL}_S$ (Section 4.1).
2. It is defined a type system $\lambda\mathbf{PL}_S$ that allows to ‘encode’ the elements (terms, formulae and derivations) of $\mathbf{PL}_S$. Then, it is proven a correspondence between derivations in $\mathbf{PL}_S$ and derivations in $\lambda\mathbf{PL}_S$ (Section 4.2).
3. The type system $\lambda\mathbf{PL}_S$ is translated into **PTT**, via a propositions-as-types interpretation, and a property of this translation is proven (Section 4.3).

⁶**PRED** is many-sorted and is minimal in the sense that only the logical constants $\supset$ and $\forall$ are considered.

4. Soundness and completeness of the propositions-as-types correspondence between PL_S and PTT are proven joining the results of the two previous items and taking advantage of the translation of PTT into ITT (Section 4.4).

Although we could have establish a propositions-as-types correspondence between PTT and PL more directly, following Martin-Löf's formalisation of intuitionistic first-order logic into intuitionistic type theory in [14], our proof have the following advantages:

- It is more general, because PL can be viewed as the particular case of PL_S with only one sort.⁷
- The concept of context in ITT is used to declare the elements of a first-order signature in the object-language of PTT , instead of defining these elements by means of a simultaneous inductive definition of types and terms in the meta-language (as it is done in [14]).
- Assumptions in first-order logic are also declared in contexts in PTT , instead of introducing these declarations in the theory of types extending in this way the theory (as it is done in [14]).

4.1 Extension of PL to PL_S

We shall now extend PL to the many-sorted logic PL_S (closely following Enderton's definition of many-sorted first-order language, terms and formulae in [5, Section 4.3]). Hereafter, we shall fix S as being a non-empty set whose elements are called *sorts*, and $S^{<\omega}$ will denote the set of finite sequences of elements of S . Following usual terminology, the elements of $S^{<\omega}$ will also be called sorts. Moreover, we assume denumerable sets of variables $V_s = \{x_1^s, x_2^s, \dots\}$ for each $s \in S$ (in some places subscripts are omitted and other letters are used for variables).

Definition 4.1.1. An S -sorted first-order language $\mathcal{L}_S$ is a structure $\langle (C_s)_{s \in S}, (F_s)_{s \in S^{<\omega}}, (P_s)_{s \in S^{<\omega}} \rangle$ where:

- (i) C_s is a (possibly empty) set of *constant symbols of sort s* , for each $s \in S$.
- (ii) F_s is a (possibly empty) set of *function symbols of sort s* , for each $s \in S^{<\omega}$. If $s = \langle s_1, \dots, s_{n+1} \rangle$, symbols in F_s represent functions from $s_1 \times \dots \times s_n$ to s_{n+1} .
- (iii) P_s is a (possibly empty) set of *predicate symbols of sort s* , for each $s \in S^{<\omega}$. If $s = \langle s_1, \dots, s_n \rangle$, symbols in P_s represent subsets of $s_1 \times \dots \times s_n$.
- (iv) The sets C_s, F_s and P_s are mutually disjoint.

Example 4.1.2. Let $S = \{A, B\}$ and let $\mathcal{L}_S = \langle (C_s)_{s \in S}, (F_s)_{s \in S^{<\omega}}, (P_s)_{s \in S^{<\omega}} \rangle$ where:

- (i) $C_A = \{c\}$ and $C_B = \emptyset$.
- (ii) $F_{\langle A, A \rangle} = \{f\}$, $F_{\langle A, B, A \rangle} = \{g\}$ and $F_s = \emptyset$ for any other $s \in S^{<\omega}$.
- (iii) $P_{\langle A \rangle} = \{P\}$, $P_{\langle A, B \rangle} = \{Q\}$ and $P_s = \emptyset$ for any other $s \in S^{<\omega}$.

Then, $\mathcal{L}_S$ is an S -sorted first-order language with a constant symbol c of sort A ; function symbols f and g of sorts $\langle A, A \rangle$ and $\langle A, B, A \rangle$, respectively; and predicate symbols P and Q of sorts $\langle A \rangle$ and $\langle A, B \rangle$, respectively.

⁷Although it is well-known that many-sorted logics can be reduced to one-sorted (or unsorted) logics, Manzano [13, Chapter VI, Section 1.2] highlighted that this reduction have a price. Thus, this generalisation is worthwhile.

Hereafter, we shall fix $\mathcal{L}_S = \langle (C_s)_{s \in S}, (F_s)_{s \in S < \omega}, (P_s)_{s \in S < \omega} \rangle$ as being an S -sorted first-order language.

Definition 4.1.3. The *sets of terms of sort s* , denoted $\mathbb{T}_s(\mathcal{L}_S)$, are inductively and simultaneously defined for all $s \in S$ by:

- (i) $V_s \subseteq \mathbb{T}_s(\mathcal{L}_S)$.
- (ii) $C_s \subseteq \mathbb{T}_s(\mathcal{L}_S)$.
- (iii) If $f \in F_{s'}$, $s' = \langle s_1, \dots, s_n, s \rangle$ and $t_i \in \mathbb{T}_{s_i}(\mathcal{L}_S)$, for $1 \leq i \leq n$, then $f(t_1, \dots, t_n) \in \mathbb{T}_s(\mathcal{L}_S)$.

Definition 4.1.4. The *set of formulae of $\mathcal{L}_S$* , denoted $\mathbb{F}(\mathcal{L}_S)$, is inductively defined by:

- (i) If $P \in P_{s'}$, $s' = \langle s_1, \dots, s_n \rangle$ and $t_i \in \mathbb{T}_{s_i}(\mathcal{L}_S)$, for $1 \leq i \leq n$, then $P(t_1, \dots, t_n) \in \mathbb{F}(\mathcal{L}_S)$.
- (ii) If $A \in \mathbb{F}(\mathcal{L}_S)$, then $\neg A \in \mathbb{F}(\mathcal{L}_S)$.
- (iii) If $A, B \in \mathbb{F}(\mathcal{L}_S)$, then $A \diamond B \in \mathbb{F}(\mathcal{L}_S)$.
- (iv) If $A \in \mathbb{F}(\mathcal{L}_S)$ and $x_i^s \in V_s$, for some $s \in S$, then $(\#x^s)A \in \mathbb{F}(\mathcal{L}_S)$.

The S -sorted paraconsistent logic PL_S , on the S -sorted first-order language $\mathcal{L}_S$, is defined by the natural deduction system that contains the same rules for propositional logical constants that PL and sorts in the quantifier rules of PL as shown in Table 3.⁸

$\frac{A(y^s/x^s)}{(\forall x^s)A} \forall\text{I}$	$\frac{(\forall x^s)A}{A(t/x^s)} \forall\text{E}$
$\frac{A(t/x^s)}{(\exists x^s)A} \exists\text{I}$	$\frac{\begin{array}{c} (A(y^s/x^s)) \\ \vdots \\ (\exists x^s)A \end{array} \quad C}{C} \exists\text{E}$
$\frac{\neg A(t/x^s)}{\neg(\forall x^s)A} \neg\forall\text{I}$	$\frac{\begin{array}{c} (\neg A(y^s/x^s)) \\ \vdots \\ \neg(\forall x^s)A \end{array} \quad C}{C} \neg\forall\text{E}$
$\frac{\neg A(y^s/x^s)}{\neg(\exists x^s)A} \neg\exists\text{I}$	$\frac{\neg(\exists x^s)A}{\neg A(t/x^s)} \neg\exists\text{E}$

Table 3: Introduction and elimination rules for quantifiers and negation of quantifiers in PL_S .

Rules for quantifiers in PL_S have analogous conditions to those mentioned in Section 1 for negation of quantifiers in PL. In addition to these conditions, in the rules $\forall\text{E}$, $\exists\text{I}$, $\neg\forall\text{I}$ and $\neg\exists\text{E}$ the term t must be of sort s , that is, $t \in \mathbb{T}_s(\mathcal{L}_S)$.

⁸Although quantifier rules in PL_S and PL are different, we use the same name for these rules in both logics taking into consideration that from the context it will be clear to which system they correspond.

4.2 The type system λPL_S and its correspondence with PL_S

Now, we shall define the type system λPL_S . In λPL_S there are the following universes of types: $*^S$, $*^P$, $*^F$ and $\square^P$; and we have that $*^P : \square^P$. Universe $*^S$ is for the sorts of PL_S , universe $*^P$ is for the propositions of PL_S , universe $*^F$ is for the sorts of first-order functions of PL_S and universe $\square^P$ is for the sorts of first-order predicates of PL_S . Formation rules in λPL_S are presented in Table 4. Formation rules $(*^S, *^S, *^F)$ and $(*^S, *^F, *^F)$ allow the construction of first-order classes of functions of any sort $s_1 \rightarrow \dots \rightarrow s_n \rightarrow s_{n+1}$, formation rule $(*^S, \square^P, \square^P)$ allows the construction of first-order classes of predicates of any sort $s_1 \rightarrow \dots \rightarrow s_n \rightarrow *^P$ and the other formation rules allow the construction of propositions. Introduction and elimination rules for type constructors $\rightarrow$, $\supset$, $\wedge$, $\vee$, $\forall$ and $\exists$ are presented in Table 5 (where $\rightsquigarrow \in \{\supset, \rightarrow\}$)⁹, and introduction and elimination rules for $\neg$ (for each one of the type constructors corresponding to logical constants) are presented in Table 6.

$\frac{A : *^S \quad B : *^S}{A \rightarrow B : *^F} (*^S, *^S, *^F)$	$\frac{A : *^S \quad B : *^F}{A \rightarrow B : *^F} (*^S, *^F, *^F)$
$\frac{A : *^S \quad B : \square^P}{A \rightarrow B : \square^P} (*^S, \square^P, \square^P)$	$\frac{A : *^P}{\neg A : *^P}$
$\frac{A : *^P \quad B : *^P}{A \diamond B : *^P}$	$\frac{\begin{array}{c} (x : A) \\ \vdots \\ A : *^S \quad B(x) : *^P \end{array}}{\#x : A.B(x) : *^P}$

Table 4: Formation rules in λPL_S .

The following definitions and lemmas are in order to prove the correspondence between derivations in PL_S and derivations in λPL_S . Hereafter, we assume that the set of sorts S is finite.

Definition 4.2.1. The *canonical context* for $\mathcal{L}_S$ in λPL_S , denoted $\Gamma_{\mathcal{L}_S}$, is a list that contains the following declarations:

- (i) For each $s \in S$, a declaration $s : *^S$.
- (ii) For each non-empty set C_s , and for each $c \in C_s$, a declaration $c : s$.
- (iii) For each non-empty set F_s , and for each $f \in F_s$, if $s = \langle s_1, \dots, s_{n+1} \rangle$, a declaration $f : s_1 \rightarrow \dots \rightarrow s_n \rightarrow s_{n+1}$.
- (iv) For each non-empty set P_s , and for each $P \in P_s$, if $s = \langle s_1, \dots, s_n \rangle$, a declaration $P : s_1 \rightarrow \dots \rightarrow s_n \rightarrow *^P$.

Example 4.2.2. For the language $\mathcal{L}_S$ of Example 4.1.2, the canonical context in λPL_S is $\Gamma_{\mathcal{L}_S} = \langle A : *^S, B : *^S, c : A, f : A \rightarrow A, g : A \rightarrow B \rightarrow A, P : A \rightarrow *^P, Q : A \rightarrow B \rightarrow *^P \rangle$.

⁹Although the type constructors $\supset$ and $\rightarrow$ have the same introduction and elimination rules in λPL_S , we use these two type constructors instead of unifying them in order to differentiate the constructor for implication $\supset$ from the constructor for allowing the construction of first-order predicates $\rightarrow$.

$\frac{(x : A) \quad \vdots \quad b(x) : B}{(\lambda x)b(x) : A \rightsquigarrow B}$	$\frac{a : A \rightsquigarrow B \quad b : A}{\mathbf{Ap}(a, b) : B}$
$\frac{a : A \quad b : B}{(a, b) : A \wedge B}$	$\frac{c : A \wedge B}{\mathbf{p}(c) : A} \quad \frac{c : A \wedge B}{\mathbf{q}(c) : B}$
$\frac{a : A}{\mathbf{i}(a) : A \vee B} \quad \frac{b : B}{\mathbf{j}(b) : A \vee B}$	$\frac{(x : A) \quad (y : B) \quad \vdots \quad \vdots \quad c : A \vee B \quad d(x) : C(\mathbf{i}(x)) \quad e(y) : C(\mathbf{j}(y))}{\mathbf{D}(c, (x)d(x), (y)e(y)) : C(c)}$
$\frac{(x : A) \quad \vdots \quad b(x) : B(x)}{(\lambda x)b(x) : (\forall x : A)B(x)}$	$\frac{b : (\forall x : A)B(x) \quad a : A}{\mathbf{Ap}(b, a) : B(a)}$
$\frac{a : A \quad b : B(a)}{(a, b) : (\exists x : A)B(x)}$	$\frac{(x : A, y : B(x)) \quad \vdots \quad c : (\exists x : A)B(x) \quad d(x, y) : C((x, y))}{\mathbf{E}(c, (x, y)d(x, y)) : C(c)}$

Table 5: Introduction and elimination rules for $\rightarrow, \supset, \wedge, \vee, \forall$ and $\exists$ in $\lambda\mathbf{PL}_S$.

Notation. We shall write $\Gamma \uplus \Gamma'$ for denoting the concatenation of the contexts Γ and Γ' and we shall write $\mathbf{Ap}(f, t_1, \dots, t_n)$ instead of $\mathbf{Ap}(\dots \mathbf{Ap}(f, t_1) \dots, t_n)$.

Definition 4.2.3. Given a term $t \in \mathbb{T}_s(\mathcal{L}_S)$, the *canonical translation of t into $\lambda\mathbf{PL}_S$* , denoted $\llbracket t \rrbracket$, and the *canonical context for t in $\lambda\mathbf{PL}_S$* , denoted Γ_t , are inductively and simultaneously defined by:

$$\begin{aligned} \llbracket x^s \rrbracket &= x^s, & \Gamma_{x^s} &= \langle x^s : s \rangle, \\ \llbracket c \rrbracket &= c, & \Gamma_c &= \langle \rangle, \\ \llbracket f(t_1, \dots, t_n) \rrbracket &= \mathbf{Ap}(f, \llbracket t_1 \rrbracket, \dots, \llbracket t_n \rrbracket), & \Gamma_{f(t_1, \dots, t_n)} &= \Gamma_{t_1} \uplus \dots \uplus \Gamma_{t_n}. \end{aligned}$$

Lemma 4.2.4. *If $t \in \mathbb{T}_s(\mathcal{L}_S)$, then $\Gamma_{\mathcal{L}_S}, \Gamma_t \vdash_{\lambda\mathbf{PL}_S} \llbracket t \rrbracket : s$.*

Proof. By structural induction on terms. □

Definition 4.2.5. Given a formula $A \in \mathbb{F}(\mathcal{L}_S)$, the *canonical translation of A into $\lambda\mathbf{PL}_S$* , denoted $\llbracket A \rrbracket$, and the *canonical context for A in $\lambda\mathbf{PL}_S$* , denoted Γ_A , are inductively and simultaneously defined by:

$$\llbracket P(t_1, \dots, t_n) \rrbracket = \mathbf{Ap}(P, \llbracket t_1 \rrbracket, \dots, \llbracket t_n \rrbracket), \quad \Gamma_{P(t_1, \dots, t_n)} = \Gamma_{t_1} \uplus \dots \uplus \Gamma_{t_n},$$

$\frac{a : A}{a : \neg\neg A}$	$\frac{a : \neg\neg A}{a : A}$
$\frac{a : A \quad b : \neg B}{(a, b) : \neg(A \supset B)}$	$\frac{c : \neg(A \supset B)}{p(c) : A} \quad \frac{c : \neg(A \supset B)}{q(c) : \neg B}$
$\frac{a : \neg A}{i(a) : \neg(A \wedge B)}$ $\frac{b : \neg B}{j(b) : \neg(A \wedge B)}$	$\frac{\begin{array}{c} (x : \neg A) \quad (y : \neg B) \\ \vdots \quad \vdots \\ c : \neg(A \wedge B) \quad d(x) : C(i(x)) \quad e(y) : C(j(y)) \end{array}}{D(c, (x)d(x), (y)e(y)) : C(c)}$
$\frac{a : \neg A \quad b : \neg B}{(a, b) : \neg(A \vee B)}$	$\frac{c : \neg(A \vee B)}{p(c) : \neg A} \quad \frac{c : \neg(A \vee B)}{q(c) : \neg B}$
$\frac{a : A \quad b : \neg B(a)}{(a, b) : \neg((\forall x : A)B(x))}$	$\frac{\begin{array}{c} (x : A, y : \neg B(x)) \\ \vdots \\ c : \neg((\forall x : A)B(x)) \quad d(x, y) : C((x, y)) \end{array}}{E(c, (x, y)d(x, y)) : C(c)}$
$\frac{\begin{array}{c} (x : A) \\ \vdots \\ b(x) : \neg B(x) \end{array}}{(\lambda x)b(x) : \neg((\exists x : A)B(x))}$	$\frac{b : \neg((\exists x : A)B(x)) \quad a : A}{Ap(b, a) : \neg B(a)}$

Table 6: Introduction and elimination rules for $\neg$ in λPL_S .

$$\begin{array}{ll} \llbracket \neg B \rrbracket = \neg \llbracket B \rrbracket, & \Gamma_{\neg B} = \Gamma_B, \\ \llbracket B \diamond C \rrbracket = \llbracket B \rrbracket \diamond \llbracket C \rrbracket, & \Gamma_{B \diamond C} = \Gamma_B \uplus \Gamma_C, \\ \llbracket \#x^s.B \rrbracket = \#x^s : s \llbracket B \rrbracket, & \Gamma_{\#x^s.B} = \Gamma_B - \langle x^s : s \rangle. \end{array}$$

Lemma 4.2.6. *If $A \in \mathbb{F}(\mathcal{L}_S)$, then $\Gamma_{\mathcal{L}_S}, \Gamma_A \vdash_{\lambda\text{PL}_S} \llbracket A \rrbracket : *^P$.*

Proof. By structural induction on formulae. □

Following Remark (iii) in [22, p. 41], if we take as convention that formulae differing only in the name of bound variables are equal, then rules $\forall\text{I}$, $\exists\text{E}$, $\neg\forall\text{E}$ and $\neg\exists\text{I}$ of PL_S can be simplified substituting $A(y^s/x^s)$ by A . In the following definition we assume such convention and the simplified rules.

Definition 4.2.7. The notion ‘ $\mathcal{D}$ is a derivation showing $\Delta \vdash_{\text{PL}_S} A$ ’, denoted $\mathcal{D} : \Delta \vdash_{\text{PL}_S} A$, is defined by the following clauses; where $\star \Rightarrow *$ means that $*$ follows from $\star$ and separation by semicolon on both sides of $\Rightarrow$ means conjunction (clauses for quantifiers are subject to the conditions previously mentioned).

- (hyp) $A \in \Delta \Rightarrow \mathcal{P}_A : \Delta \vdash_{\text{PL}_5} A$.
- ($\supset$) i. $\mathcal{D}' : \Delta, B \vdash_{\text{PL}_5} C \Rightarrow \mathcal{I}^\supset[B](\mathcal{D}') : \Delta \vdash_{\text{PL}_5} B \supset C$.
 ii. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} B \supset C; \mathcal{D}'' : \Delta \vdash_{\text{PL}_5} B \Rightarrow \mathcal{E}^\supset(\mathcal{D}', \mathcal{D}'') : \Delta \vdash_{\text{PL}_5} C$.
- ($\wedge$) i. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} B; \mathcal{D}'' : \Delta \vdash_{\text{PL}_5} C \Rightarrow \mathcal{I}^\wedge(\mathcal{D}', \mathcal{D}'') : \Delta \vdash_{\text{PL}_5} B \wedge C$.
 ii. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} B \wedge C \Rightarrow \mathcal{E}_1^\wedge(\mathcal{D}') : \Delta \vdash_{\text{PL}_5} B; \mathcal{E}_2^\wedge(\mathcal{D}') : \Delta \vdash_{\text{PL}_5} C$.
- ($\vee$) i. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} B \Rightarrow \mathcal{I}_1^\vee[C](\mathcal{D}') : \Delta \vdash_{\text{PL}_5} B \vee C; \mathcal{I}_2^\vee[C](\mathcal{D}') : \Delta \vdash_{\text{PL}_5} C \vee B$.
 ii. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} B \vee C; \mathcal{D}'' : \Delta, B \vdash_{\text{PL}_5} A; \mathcal{D}''' : \Delta, C \vdash_{\text{PL}_5} A \Rightarrow \mathcal{E}^\vee[B, C](\mathcal{D}', \mathcal{D}'', \mathcal{D}''') : \Delta \vdash_{\text{PL}_5} A$.
- ($\forall$) i. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} B \Rightarrow \mathcal{I}^\forall[x^s](\mathcal{D}') : \Delta \vdash_{\text{PL}_5} (\forall x^s)B$.
 ii. $t \in \mathbb{T}_s(\mathcal{L}_S); \mathcal{D}' : \Delta \vdash_{\text{PL}_5} (\forall x^s)B \Rightarrow \mathcal{E}^\forall[t](\mathcal{D}') : \Delta \vdash_{\text{PL}_5} B(t/x^s)$.
- ($\exists$) i. $t \in \mathbb{T}_s(\mathcal{L}_S); \mathcal{D}' : \Delta \vdash_{\text{PL}_5} B(t/x^s) \Rightarrow \mathcal{I}^\exists[t, x^s](\mathcal{D}') : \Delta \vdash_{\text{PL}_5} (\exists x^s)B$.
 ii. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} (\exists x^s)B; \mathcal{D}'' : \Delta, B \vdash_{\text{PL}_5} A \Rightarrow \mathcal{E}^\exists[B, x^s](\mathcal{D}', \mathcal{D}'') : \Delta \vdash_{\text{PL}_5} A$.
- ($\neg\neg$) i. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} B \Rightarrow \mathcal{I}^{\neg\neg}(\mathcal{D}') : \Delta \vdash_{\text{PL}_5} \neg\neg B$.
 ii. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} \neg\neg B \Rightarrow \mathcal{E}^{\neg\neg}(\mathcal{D}') : \Delta \vdash_{\text{PL}_5} B$.
- ($\neg\supset$) i. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} B; \mathcal{D}'' : \Delta \vdash_{\text{PL}_5} \neg C \Rightarrow \mathcal{I}^{\neg\supset}(\mathcal{D}', \mathcal{D}'') : \Delta \vdash_{\text{PL}_5} \neg(B \supset C)$.
 ii. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} \neg(B \supset C) \Rightarrow \mathcal{E}_1^{\neg\supset}(\mathcal{D}') : \Delta \vdash_{\text{PL}_5} B; \mathcal{E}_2^{\neg\supset}(\mathcal{D}') : \Delta \vdash_{\text{PL}_5} \neg C$.
- ($\neg\wedge$) i. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} \neg B \Rightarrow \mathcal{I}_1^{\neg\wedge}[C](\mathcal{D}') : \Delta \vdash_{\text{PL}_5} \neg(B \wedge C); \mathcal{I}_2^{\neg\wedge}[C](\mathcal{D}') : \Delta \vdash_{\text{PL}_5} \neg(C \wedge B)$.
 ii. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} \neg(B \wedge C); \mathcal{D}'' : \Delta, \neg B \vdash_{\text{PL}_5} A; \mathcal{D}''' : \Delta, \neg C \vdash_{\text{PL}_5} A \Rightarrow \mathcal{E}^{\neg\wedge}[B, C](\mathcal{D}', \mathcal{D}'', \mathcal{D}''') : \Delta \vdash_{\text{PL}_5} A$.
- ($\neg\vee$) i. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} \neg B; \mathcal{D}'' : \Delta \vdash_{\text{PL}_5} \neg C \Rightarrow \mathcal{I}^{\neg\vee}(\mathcal{D}', \mathcal{D}'') : \Delta \vdash_{\text{PL}_5} \neg(B \vee C)$.
 ii. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} \neg(B \vee C) \Rightarrow \mathcal{E}_1^{\neg\vee}(\mathcal{D}') : \Delta \vdash_{\text{PL}_5} \neg B; \mathcal{E}_2^{\neg\vee}(\mathcal{D}') : \Delta \vdash_{\text{PL}_5} \neg C$.
- ($\neg\forall$) i. $t \in \mathbb{T}_s(\mathcal{L}_S); \mathcal{D}' : \Delta \vdash_{\text{PL}_5} \neg B(t/x^s) \Rightarrow \mathcal{I}^{\neg\forall}[t, x^s](\mathcal{D}') : \Delta \vdash_{\text{PL}_5} \neg(\forall x^s)B$.
 ii. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} \neg(\forall x^s)B; \mathcal{D}'' : \Delta, \neg B \vdash_{\text{PL}_5} A \Rightarrow \mathcal{E}^{\neg\forall}[B, x^s](\mathcal{D}', \mathcal{D}'') : \Delta \vdash_{\text{PL}_5} A$.
- ($\neg\exists$) i. $\mathcal{D}' : \Delta \vdash_{\text{PL}_5} \neg B \Rightarrow \mathcal{I}^{\neg\exists}[x^s](\mathcal{D}') : \Delta \vdash_{\text{PL}_5} \neg(\exists x^s)B$.
 ii. $t \in \mathbb{T}_s(\mathcal{L}_S); \mathcal{D}' : \Delta \vdash_{\text{PL}_5} \neg(\exists x^s)B \Rightarrow \mathcal{E}^{\neg\exists}[t](\mathcal{D}') : \Delta \vdash_{\text{PL}_5} \neg B(t/x^s)$.

The intended interpretation of symbols in derivations is: P_A stands for ‘assumption of A ’; $\mathcal{I}^\square[\text{pars}](\mathcal{D})$ stands for the derivation obtained by applying the introduction rule for the logical constant(s) $\square$ to the root of deduction $\mathcal{D}$ with parameters pars (some apparently unnecessary parameters are added in some cases to enable the translation of derivations in PL_5 into terms of λPL_5); and $\mathcal{E}^\square[\text{pars}](\mathcal{D})$ have an analogous meaning of $\mathcal{I}^\square[\text{pars}](\mathcal{D})$, but it corresponds to elimination rules.

Definition 4.2.8. Let $\Delta = \{A_1, \dots, A_n\}$ be a subset of $\mathbb{F}(\mathcal{L}_S)$, the *canonical translation of Δ into λPL_S* , denoted Γ_Δ , is the context defined by:

$$\Gamma_\Delta = \Gamma_{A_1} \# \dots \# \Gamma_{A_n} \# \langle x_{A_1} : \llbracket A_1 \rrbracket, \dots, x_{A_n} : \llbracket A_n \rrbracket \rangle.$$

Definition 4.2.9. For $\mathcal{D} : \Delta \vdash_{\text{PL}_S} A$, the *canonical translation of $\mathcal{D}$ into λPL_S* , denoted $\llbracket \mathcal{D} \rrbracket$, and the *canonical context for $\mathcal{D}$ in λPL_S* , denoted $\Gamma_{\mathcal{D}}$, are inductively and simultaneously defined by:

$$\begin{aligned} \llbracket \mathcal{P}_A \rrbracket &= x_A, & \Gamma_{\mathcal{D}} &= \langle x_A : \llbracket A \rrbracket \rangle, \\ \llbracket \mathcal{I}^\supset[B](\mathcal{D}') \rrbracket &= (\lambda x_B) \llbracket \mathcal{D}' \rrbracket, & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'} - \langle x_B : \llbracket B \rrbracket \rangle, \\ \llbracket \mathcal{E}^\supset(\mathcal{D}', \mathcal{D}'') \rrbracket &= \text{Ap}(\llbracket \mathcal{D}' \rrbracket, \llbracket \mathcal{D}'' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'} \# \Gamma_{\mathcal{D}''}, \\ \llbracket \mathcal{I}^\wedge(\mathcal{D}', \mathcal{D}'') \rrbracket &= (\llbracket \mathcal{D}' \rrbracket, \llbracket \mathcal{D}'' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'} \# \Gamma_{\mathcal{D}''}, \\ \llbracket \mathcal{E}_1^\wedge(\mathcal{D}') \rrbracket &= \text{p}(\llbracket \mathcal{D}' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \\ \llbracket \mathcal{E}_2^\wedge(\mathcal{D}') \rrbracket &= \text{q}(\llbracket \mathcal{D}' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \\ \llbracket \mathcal{I}_1^\vee[C](\mathcal{D}') \rrbracket &= \text{i}(\llbracket \mathcal{D}' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \\ \llbracket \mathcal{I}_2^\vee[C](\mathcal{D}') \rrbracket &= \text{j}(\llbracket \mathcal{D}' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \\ \llbracket \mathcal{E}^\vee[B, C](\mathcal{D}', \mathcal{D}'', \mathcal{D}''') \rrbracket &= \text{D}(\llbracket \mathcal{D}' \rrbracket, (x_B) \llbracket \mathcal{D}'' \rrbracket (x_B), (x_C) \llbracket \mathcal{D}''' \rrbracket (x_C)), \\ & & \Gamma_{\mathcal{D}} &= (\Gamma_{\mathcal{D}'} \# \Gamma_{\mathcal{D}''} \# \Gamma_{\mathcal{D}'''}) - \langle x_B : \llbracket B \rrbracket, x_C : \llbracket C \rrbracket \rangle, \\ \llbracket \mathcal{I}^\forall[x^s](\mathcal{D}') \rrbracket &= (\lambda x^s) \llbracket \mathcal{D}' \rrbracket, & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'} - \langle x^s : s \rangle, \\ \llbracket \mathcal{E}^\forall[t](\mathcal{D}') \rrbracket &= \text{Ap}(\llbracket \mathcal{D}' \rrbracket, \llbracket t \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'} \# \Gamma_t, \\ \llbracket \mathcal{I}^\exists[t, x^s](\mathcal{D}') \rrbracket &= (\llbracket t \rrbracket, \llbracket \mathcal{D}' \rrbracket), & \Gamma_{\mathcal{D}} &= (\Gamma_{\mathcal{D}'} \# \Gamma_t) - \langle x^s : s \rangle, \\ \llbracket \mathcal{E}^\exists[B, x^s](\mathcal{D}', \mathcal{D}'') \rrbracket &= \text{E}(\llbracket \mathcal{D}' \rrbracket, (x^s, y_B) \llbracket \mathcal{D}'' \rrbracket (x^s, y_B)), \\ & & \Gamma_{\mathcal{D}} &= (\Gamma_{\mathcal{D}'} \# \Gamma_{\mathcal{D}''}) - \langle x^s : s, y_B : \llbracket B \rrbracket \rangle, \\ \llbracket \mathcal{I}^{\neg\supset}(\mathcal{D}') \rrbracket &= \llbracket \mathcal{D}' \rrbracket, & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \\ \llbracket \mathcal{E}^{\neg\supset}(\mathcal{D}') \rrbracket &= \llbracket \mathcal{D}' \rrbracket, & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \\ \llbracket \mathcal{I}^{\neg\supset}(\mathcal{D}', \mathcal{D}'') \rrbracket &= (\llbracket \mathcal{D}' \rrbracket, \llbracket \mathcal{D}'' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'} \# \Gamma_{\mathcal{D}''}, \\ \llbracket \mathcal{E}_1^{\neg\supset}(\mathcal{D}') \rrbracket &= \text{p}(\llbracket \mathcal{D}' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \\ \llbracket \mathcal{E}_2^{\neg\supset}(\mathcal{D}') \rrbracket &= \text{q}(\llbracket \mathcal{D}' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \\ \llbracket \mathcal{I}_1^{\neg\wedge}[C](\mathcal{D}') \rrbracket &= \text{i}(\llbracket \mathcal{D}' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \\ \llbracket \mathcal{I}_2^{\neg\wedge}[C](\mathcal{D}') \rrbracket &= \text{j}(\llbracket \mathcal{D}' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \\ \llbracket \mathcal{E}^{\neg\wedge}[B, C](\mathcal{D}', \mathcal{D}'', \mathcal{D}''') \rrbracket &= \text{D}(\llbracket \mathcal{D}' \rrbracket, (x_{\neg B}) \llbracket \mathcal{D}'' \rrbracket (x_{\neg B}), (x_{\neg C}) \llbracket \mathcal{D}''' \rrbracket (x_{\neg C})), \\ & & \Gamma_{\mathcal{D}} &= (\Gamma_{\mathcal{D}'} \# \Gamma_{\mathcal{D}''} \# \Gamma_{\mathcal{D}'''}) - \langle x_{\neg B} : \llbracket \neg B \rrbracket, x_{\neg C} : \llbracket \neg C \rrbracket \rangle, \\ \llbracket \mathcal{I}^{\neg\vee}(\mathcal{D}', \mathcal{D}'') \rrbracket &= (\llbracket \mathcal{D}' \rrbracket, \llbracket \mathcal{D}'' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'} \# \Gamma_{\mathcal{D}''}, \\ \llbracket \mathcal{E}_1^{\neg\vee}(\mathcal{D}') \rrbracket &= \text{p}(\llbracket \mathcal{D}' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \\ \llbracket \mathcal{E}_2^{\neg\vee}(\mathcal{D}') \rrbracket &= \text{q}(\llbracket \mathcal{D}' \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'}, \end{aligned}$$

$$\llbracket \mathcal{I}^{-\forall}[t, x^s](\mathcal{D}') \rrbracket = (\llbracket t \rrbracket, \llbracket \mathcal{D}' \rrbracket), \quad \Gamma_{\mathcal{D}} = (\Gamma_{\mathcal{D}'} \uplus \Gamma_t) - \langle x^s : s \rangle,$$

$$\begin{aligned} \llbracket \mathcal{E}^{-\forall}[B, x^s](\mathcal{D}', \mathcal{D}'') \rrbracket &= \mathbf{E}(\llbracket \mathcal{D}' \rrbracket, (x^s, y_{\neg B}) \llbracket \mathcal{D}'' \rrbracket (x^s, y_{\neg B})), \\ \Gamma_{\mathcal{D}} &= (\Gamma_{\mathcal{D}'} \uplus \Gamma_{\mathcal{D}''}) - \langle x^s : s, y_{\neg B} : \llbracket \neg B \rrbracket \rangle, \end{aligned}$$

$$\begin{aligned} \llbracket \mathcal{I}^{-\exists}[x^s](\mathcal{D}') \rrbracket &= (\lambda x^s) \llbracket \mathcal{D}' \rrbracket, & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'} - \langle x^s : s \rangle, \\ \llbracket \mathcal{E}^{-\exists}[t](\mathcal{D}') \rrbracket &= \mathbf{Ap}(\llbracket \mathcal{D}' \rrbracket, \llbracket t \rrbracket), & \Gamma_{\mathcal{D}} &= \Gamma_{\mathcal{D}'} \uplus \Gamma_t. \end{aligned}$$

Lemma 4.2.10. *If $\mathcal{D} : \Delta \vdash_{\text{PL}_5} A$, then $\Gamma_{\mathcal{L}_S}, \Gamma_{\Delta}, \Gamma_A, \Gamma_{\mathcal{D}} \vdash_{\lambda\text{PL}_5} \llbracket \mathcal{D} \rrbracket : \llbracket A \rrbracket$.*

Proof. By induction on the derivation in PL_5 . □

Note that in the canonical contexts all declarations are of one of the following forms:

$$\begin{aligned} s : *^S, & & (\text{declaration of sort}); \\ x : s, & & (\text{declaration of variable of sort } s); \\ c : s, & & (\text{declaration of constant of sort } s); \\ f : s_1 \rightarrow \cdots \rightarrow s_n \rightarrow s_{n+1}, & & (\text{declaration of first-order function}); \\ P : s_1 \rightarrow \cdots \rightarrow s_n \rightarrow *^P, & & (\text{declaration of first-order predicate}); \\ x_A : \llbracket A \rrbracket, \text{ where } \Gamma_{\mathcal{L}_S}, \Gamma_A \vdash_{\lambda\text{PL}_5} \llbracket A \rrbracket : *^P, & & (\text{assumption of proposition } A). \end{aligned}$$

Hereafter, we shall assume that contexts in λPL_5 have only this kind of declarations.

Definition 4.2.11. Let Γ be a context of λPL_5 . The *set of sorts associated with Γ* is $S_{\Gamma} = \{s \mid s : *^S \in \Gamma\}$ and the *S_{Γ} -sorted first-order language associated with Γ* is $\mathcal{L}_{S_{\Gamma}} = \langle (C_s)_{s \in S}, (F_s)_{s \in S^{<\omega}}, (P_s)_{s \in S^{<\omega}} \rangle$, where:

$$\begin{aligned} C_s &= \{c \mid c : s \in \Gamma\}, \text{ for each } s \in S, \\ F_{\langle s_1, \dots, s_{n+1} \rangle} &= \{f \mid f : s_1 \rightarrow \cdots \rightarrow s_n \rightarrow s_{n+1} \in \Gamma\}, \\ P_{\langle s_1, \dots, s_n \rangle} &= \{P \mid P : s_1 \rightarrow \cdots \rightarrow s_n \rightarrow *^P \in \Gamma\}. \end{aligned}$$

Notation. Hereafter, we shall use $\Gamma \vdash_{\lambda\text{PL}_5} t : A : U$, for some context Γ , term t , type A and universe U , to denote $\Gamma \vdash_{\lambda\text{PL}_5} A : U$ and $\Gamma \vdash_{\lambda\text{PL}_5} t : A$.

Definition 4.2.12. If $\Gamma \vdash_{\lambda\text{PL}_5} t : s : *^S$, the *canonical inverse translation of t into $\mathbb{T}(\mathcal{L}_{S_{\Gamma}})$* , denoted $\llbracket t \rrbracket^{-1}$, is inductively defined on the derivation in λPL_5 by:

- (i) If $t = x$ and $x : s \in \Gamma$: $\llbracket t \rrbracket^{-1} = x^s$.
- (ii) If $t = c$ and $c : s \in \Gamma$: $\llbracket t \rrbracket^{-1} = c$.
- (iii) If $t = \mathbf{Ap}(f, \llbracket t_1 \rrbracket, \dots, \llbracket t_n \rrbracket)$: then $\Gamma \vdash_{\lambda\text{PL}_5} f : s_1 \rightarrow \cdots \rightarrow s_n \rightarrow s$, $\Gamma \vdash_{\lambda\text{PL}_5} t_1 : s_1 : *^S, \dots, \Gamma \vdash_{\lambda\text{PL}_5} t_n : s_n : *^S$ and $\llbracket t \rrbracket^{-1} = f(\llbracket t_1 \rrbracket^{-1}, \dots, \llbracket t_n \rrbracket^{-1})$.

Definition 4.2.13. If $\Gamma \vdash_{\lambda\text{PL}_5} A : *^P$, the *canonical inverse translation of A into $\mathbb{F}(\mathcal{L}_{S_{\Gamma}})$* , denoted $\llbracket A \rrbracket^{-1}$, is inductively defined on the derivation in λPL_5 by:

- (i) If $A = \text{Ap}(P, \llbracket t_1 \rrbracket, \dots, \llbracket t_n \rrbracket)$: then $\Gamma \vdash_{\lambda\text{PL}_S} P : s_1 \rightarrow \dots \rightarrow s_n \rightarrow *^P$, $\Gamma \vdash_{\lambda\text{PL}_S} t_1 : s_1, \dots$, $\Gamma \vdash_{\lambda\text{PL}_S} t_n : s_n$ and $\llbracket A \rrbracket^{-1} = P(\llbracket t_1 \rrbracket^{-1}, \dots, \llbracket t_n \rrbracket^{-1})$.
- (ii) If $A = \neg B$: then $\Gamma \vdash_{\lambda\text{PL}_S} B : *^P$ and $\llbracket A \rrbracket^{-1} = \neg \llbracket B \rrbracket^{-1}$.
- (iii) If $A = B \diamond C$: then $\Gamma \vdash_{\lambda\text{PL}_S} B : *^P$, $\Gamma \vdash_{\lambda\text{PL}_S} C : *^P$ and $\llbracket A \rrbracket^{-1} = \llbracket B \rrbracket^{-1} \diamond \llbracket C \rrbracket^{-1}$.
- (iv) If $A = \#x : s.B$: then $\Gamma \vdash_{\lambda\text{PL}_S} B : *^P$ and $\llbracket A \rrbracket^{-1} = \#x^s. \llbracket B \rrbracket^{-1}$.

Definition 4.2.14. Given a context Γ of λPL_S , the *set of propositional hypothesis in Γ* , denoted P_Γ , is defined by:

$$P_\Gamma = \{a : A \in \Gamma \mid \Gamma \vdash_{\lambda\text{PL}_S} A : *^P\}.$$

Definition 4.2.15. Given a context Γ of λPL_S , the *inverse translation of Γ* , denoted $\llbracket \Gamma \rrbracket^{-1}$, is the set

$$\llbracket \Gamma \rrbracket^{-1} = \{\llbracket A \rrbracket^{-1} \mid a : A \in P_\Gamma\}.$$

Lemma 4.2.16. If $\Gamma \vdash_{\lambda\text{PL}_S} a : A : *^P$, then there is a derivation $\mathcal{D}$ such that:

$$\mathcal{D} : \llbracket \Gamma \rrbracket^{-1} \vdash_{\text{PL}_S} \llbracket A \rrbracket^{-1}.$$

Moreover, $\Gamma = \Gamma_{\mathcal{L}_S} \# \Gamma_{\llbracket \Gamma \rrbracket^{-1}} \# \Gamma_{\llbracket A \rrbracket^{-1}} \# \Gamma_{\mathcal{D}}$ and $A = \llbracket \llbracket A \rrbracket^{-1} \rrbracket$.

Proof. By induction on the derivation of $\Gamma \vdash_{\lambda\text{PL}_S} a : A$. □

As it is pointed out by Barendregt [2, p. 146], there are formulae A such that

$$\vdash_{\text{PRED}} A \not\Rightarrow (\exists a)(\Gamma_{\mathcal{A}} \vdash_{\lambda\text{PRED}} a : \llbracket A \rrbracket),$$

where $\Gamma_{\mathcal{A}}$ is the canonical context for the language of **PRED**. For instance, for a sentence $A = (\forall x^s)(P(x^s) \supset Q) \supset (((\forall x^s)P(x^s)) \supset Q)$, we have that $\vdash_{\text{PRED}} A$ and there is no a such that $\Gamma_{\mathcal{A}} \vdash_{\lambda\text{PRED}} a : \llbracket A \rrbracket$. However, there is a such that $\Gamma_{\mathcal{A}}, e_s : s \vdash_{\lambda\text{PRED}} a : \llbracket A \rrbracket$. The reason of this is that logic systems (particularly **IL** and **PL**) assumes that domains of interpretation are non-empty, while in type theory it is not the case.¹⁰ In fact, the sentence A above is valid only if the sort s is interpreted as a non-empty set. Exactly the same phenomena occurs between **PL_S** and **λPL_S**, and in order to avoid this fine difference between **PL_S** and **λPL_S**, we shall follow the same strategy taken in [2], which is to extend the canonical context for $\mathcal{L}_S$ by adding declarations stating the existence of elements in each one of the sorts in S .

Definition 4.2.17. The *extended context for $\mathcal{L}_S$* is $\Gamma_{\mathcal{L}_S}^+ = \Gamma_{\mathcal{L}_S} \# \langle e_s : s \mid s \in S \rangle$.

Theorem 4.2.18. Let $\Delta \cup \{A\} \subseteq \mathbb{F}(\mathcal{L}_S)$, then:

- (i) $\mathcal{D} : \Delta \vdash_{\text{PL}_S} A$ if and only if $\Gamma_{\mathcal{L}_S} \# \Gamma_{\Delta} \# \Gamma_A \# \Gamma_{\mathcal{D}} \vdash_{\lambda\text{PL}_S} \llbracket \mathcal{D} \rrbracket : \llbracket A \rrbracket$.
- (ii) $\Delta \vdash_{\text{PL}_S} A$ if and only if $\Gamma_{\mathcal{L}_S}^+ \# \Gamma_{\Delta} \# \Gamma_A \vdash_{\lambda\text{PL}_S} a : \llbracket A \rrbracket$, for some term a .

¹⁰This difference between the logic systems and the type theories that we are considering does not show a weakness of type theory, on the contrary, it shows a strength of the type theory systems with respect to these logical systems, since the type theory systems allows us to formalise both logics that presuppose non-empty domains and logics that do not presuppose this.

Proof. (i). By Lemma 4.2.10, Lemma 4.2.16 and the fact that the canonical translation $\llbracket \cdot \rrbracket$ of formulae of PL_S into types of λPL_S is an injective mapping. Note that in the right-to-left direction, the S -sorted first-order language associated with $\Gamma_{\mathcal{L}_S} \uplus \Gamma_\Delta \uplus \Gamma_A \uplus \Gamma_{\mathcal{D}}$ can be replaced by $\mathcal{L}_S$ without any problem.

(ii). We can assume, without loss of generality, that every variable used in a derivation of $\Delta \vdash_{\text{PL}_S} A$ is (free or bound) in some formula in $\Delta \cup \{A\}$. Under this assumption, for every derivation $\mathcal{D}$ of $\Delta \vdash_{\text{PL}_S} A$, we have that $\Gamma_{\mathcal{D}} - (\Gamma_\Delta \uplus \Gamma_A)$ only has declarations of the form $x^s : s$. Moreover, we have that $\Gamma_{\mathcal{L}_S}^+ \vdash_{\lambda\text{PL}_S} e_s : s$, for each $s \in S$. Consequently,

$$\begin{aligned} \Delta \vdash_{\text{PL}_S} A &\Leftrightarrow (\exists \mathcal{D})(\mathcal{D} : \Delta \vdash_{\text{PL}_S} A) \\ &\Leftrightarrow (\exists \mathcal{D})(\Gamma_{\mathcal{L}_S} \uplus \Gamma_\Delta \uplus \Gamma_A \uplus \Gamma_{\mathcal{D}} \vdash_{\lambda\text{PL}_S} \llbracket \mathcal{D} \rrbracket : \llbracket A \rrbracket) \\ &\Leftrightarrow (\exists a)(\Gamma_{\mathcal{L}_S}^+ \uplus \Gamma_\Delta \uplus \Gamma_A \vdash_{\lambda\text{PL}_S} a : \llbracket A \rrbracket). \end{aligned}$$

For the last $\Rightarrow$ take a as the term obtained by substituting in $\llbracket \mathcal{D} \rrbracket$ each occurrence of x^s by e_s . The last $\Leftarrow$ follows from Lemma 4.2.16 (here, the S -sorted first-order language associated with $\Gamma_{\mathcal{L}_S}^+ \uplus \Gamma_\Delta$ can also be replaced by $\mathcal{L}_S$ without any problem). $\square$

4.3 The translation of λPL_S into PTT

Now, we shall show how λPL_S can be interpreted into PTT.

Definition 4.3.1 (Propositions-as-types interpretation). The *translation of an expression e of λPL_S into an expression of PTT*, denoted $|e|$, is inductively defined by:

$$\begin{aligned} |*^S| &= |*^P| = |*^F| = \text{U}_0, & |\Box^P| &= \text{U}_1, \\ |\neg e'| &= \overline{|e'|}, & |e_1 \rightsquigarrow e_2| &= |e_1| \rightarrow |e_2|, \\ |e_1 \wedge e_2| &= |e_1| \times |e_2|, & |e_1 \vee e_2| &= |e_1| + |e_2|, \\ |(\forall x : s)e'| &= (\Pi x : s)|e'|, & |(\exists x : s)e'| &= (\Sigma x : s)|e'|, \\ \text{otherwise, } |e'| &= e', \end{aligned}$$

where $\rightsquigarrow \in \{\supset, \rightarrow\}$.

Notation. For a context $\Gamma = \langle x_1 : A_1, \dots, x_n : A_n \rangle$ of λPL_S , we shall write $|\Gamma|$ for $\langle x_1 : |A_1|, \dots, x_n : |A_n| \rangle$.

Theorem 4.3.2. *If $\Gamma \vdash_{\lambda\text{PL}_S} a : A$, then $|\Gamma| \vdash_{\text{PTT}} |a| : |A|$.*

Proof. By induction on the derivation in λPL_S . $\square$

4.4 Soundness and completeness of the propositions-as-types correspondence between PL_S and PTT

The *soundness* of our propositions-as-types correspondence between PL_S and PTT is an immediate consequence of Theorem 4.2.18 and Theorem 4.3.2.

Notation. For terms $t \in \mathbb{T}_s(\mathcal{L}_S)$, formulae $A \in \mathbb{F}(\mathcal{L}_S)$ and set of formulae $\Delta \subseteq \mathbb{F}(\mathcal{L}_S)$, we shall write

$$[t] \text{ for } \llbracket t \rrbracket,$$

$$[A] \text{ for } \llbracket A \rrbracket, \text{ and}$$

$$[\Delta_A] \text{ for } |\Gamma_{\mathcal{L}_S}^+| \# |\Gamma_\Delta| \# |\Gamma_A|.$$

Theorem 4.4.1 (Soundness of the propositions-as-types correspondence). *Let $\Delta \cup \{A\}$ be a finite subset of $\mathbb{F}(\mathcal{L}_S)$. If $\Delta \vdash_{\text{PL}_S} A$, then $[\Delta_A] \vdash_{\text{PTT}} a : [A]$, for some term a .*

Now, we shall address the *completeness* of our propositions-as-types correspondence between PL_S and PTT . Via the translation function $|\cdot|$, the system λPL_S can be viewed as a fragment of PTT (where some universes collapse). Thus, proving completeness consists of proving that PTT is a *conservative extension* of λPL_S .

As it is pointed out by Geuvers and Barendsen [6], the completeness of PRED with respect to λP is an ‘intricate problem’.¹¹ The same difficulties, and some other (because we are dealing with more logical constants and type constructors), must be addressed in a direct proof of the completeness of the propositions-as-types correspondence between PL_S and PTT . However, taking advantage of the translation of PTT into ITT shown in Section 3, we shall demonstrate the completeness using a consequence of the results in [6] and [21] described in the next paragraph.

Swan [21, Section 3] defined a type system that codifies many-sorted first-order intuitionistic logic (in a similar way as λPL_S codifies PL_S). He also proved that a version of intuitionistic (or Martin-Löf) type theory with Π, Σ and $+$ types, but without universes, is conservative over that type system. The result is proven considering weak elimination rules for Σ and $+$. On the other hand, in [6], it is proven that λP is conservative over λPRED (a type system that codifies PRED , also in a similar way as λPL_S codifies PL_S). In λP there are two universes, **Set** and **Type** (with $\text{Set} : \text{Type}$), and the only type constructor is Π . The system ITT can be seen as the extension of λP with Σ and $+$ types in **Set**. Thus, completeness of many-sorted first-order intuitionistic logic without negation, which we shall denote by IL_S^- , with respect to ITT is a consequence of the results in [6] and [21].

Definition 4.4.2. The S -sorted first-order language $\mathcal{L}_S^o$ is the result of adding to $\mathcal{L}_S$ a predicate symbol P^o for each predicate symbol P in $\mathcal{L}_S$ (P^o will be of the same sort of P).

The terms and formulae generated by $\mathcal{L}_S^o$ are defined just in the same way as in Definitions 4.1.3 and 4.1.4, respectively, but without including negation.

Notation. We shall use $\mathbb{F}(\mathcal{L}_S^o)$ to denote the set of formulae generated by $\mathcal{L}_S^o$. Given $A \in \mathbb{F}(\mathcal{L}_S)$ in negative normal form, we shall use A^* to denote the result of substituting in A sub-formulae of the form $\neg P(t_1, \dots, t_n)$ by $P^o(t_1, \dots, t_n)$. For a set $\Delta \subseteq \mathbb{F}(\mathcal{L}_S)$, we shall write Δ^* for $\{A^* \mid A \in \Delta\}$.

Hereafter, we shall fix $\mathbb{F}(\mathcal{L}_S^o)$ as the set of formulae of IL_S^- .

Definition 4.4.3. The *inverse propositions-as-types interpretation*, from types of ITT into $\mathbb{F}(\mathcal{L}_S^o)$, which we shall denote by $\llbracket \cdot \rrbracket^{-1}$ (where $\cdot$ is a type of ITT that corresponds with a logic formula or a term), is inductively defined by:

$$\llbracket X \rrbracket^{-1} = X, \quad \llbracket \text{Ap}(X, t_1, \dots, t_n) \rrbracket^{-1} = X(\llbracket t_1 \rrbracket^{-1}, \dots, \llbracket t_n \rrbracket^{-1}),$$

¹¹The difficulties are basically because in λP there are several types that do not represent any sort or formula of PRED , and it is necessary to assure that these ‘meaningless’ types do not spoil the completeness. Moreover, some characteristics of λP , as the possibility of defining new predicates by abstractions and the possibility of defining higher-order functions, increases the difficulty in proving that this type system does not allow us to prove more propositions than what can be proven in PRED .

$$\begin{aligned}
\|A \rightarrow B\|^{-1} &= \|A\|^{-1} \supset \|B\|^{-1}, & \|A \times B\|^{-1} &= \|A\|^{-1} \wedge \|B\|^{-1}, \\
\|A + B\|^{-1} &= \|A\|^{-1} \vee \|B\|^{-1}, & \|(\Pi x : s)A\|^{-1} &= (\forall x^s)\|A\|^{-1}, \\
\|(\Sigma x : s)A\|^{-1} &= (\exists x^s)\|A\|^{-1},
\end{aligned}$$

where the meta-variable X ranges over term and type variables.

The definition of negative normal form (Definition 1.7), which applies to formulae of $\mathbf{PL}$, can be adapted to apply to formulae of $\mathbf{PL}_S$ just by adding sorts in the quantified variables. In the rest of this section, we assume this adaptation.

Lemma 4.4.4. *Let $A \in \mathbb{F}(\mathcal{L}_S)$. Then $\|H([A])\|^{-1}$ and $\text{NNF}(A)^*$ are the same formula.*

Proof. By structural induction on A . □

Lemma 4.4.5. *Let $\Delta \cup \{A\} \subseteq \mathbb{F}(\mathcal{L}_S)$. If $\text{NNF}(\Delta)^* \vdash_{\mathbf{IL}_S^-} \text{NNF}(A)^*$, then $\Delta \vdash_{\mathbf{PL}_S} A$.*

Proof. Let suppose that $\text{NNF}(\Delta)^* \vdash_{\mathbf{IL}_S^-} \text{NNF}(A)^*$. By induction on the derivation, it can be proven that $\text{NNF}(\Delta) \vdash_{\mathbf{PL}_S} \text{NNF}(A)$. Then, by Theorem 1.9 (which is also valid in the extension $\mathbf{PL}_S$), it follows that $\Delta \vdash_{\mathbf{PL}_S} A$. □

Theorem 4.4.6 (Completeness of the propositions-as-types correspondence). *Let $\Delta \cup \{A\}$ be a finite subset of $\mathbb{F}(\mathcal{L}_S)$. If $[\Delta_A] \vdash_{\mathbf{PTT}} a : [A]$, for some term a , then $\Delta \vdash_{\mathbf{PL}_S} A$.*

Proof. Let suppose that $[\Delta_A] \vdash_{\mathbf{PTT}} a : [A]$. By Theorem 3.2, it follows that $H([\Delta_A]) \vdash_{\mathbf{ITT}} H(a) : H([A])$. Then, by completeness of $\mathbf{IL}_S^-$ with respect to $\mathbf{ITT}$ and Lemma 4.4.4, we have that $\text{NNF}(\Delta)^* \vdash_{\mathbf{IL}_S^-} \text{NNF}(A)^*$. Consequently, $\Delta \vdash_{\mathbf{PL}_S} A$ by Lemma 4.4.5. □

5 Why PTT is paraconsistent

It is said that a type theory is *inconsistent* if all types corresponding to logic formulae—under some propositions-as-types interpretation—are inhabited (see, for instance, [4]). It is because inhabitation of these types correspond to provability of the respective formulae in the corresponding logic. Thus, inhabitation of all types corresponding to formulae is equivalent to provability of all formulae in the logic; we shall call a formal system with this property *logically trivial*. In type systems where there is a type $\perp$ that codifies the bottom particle of logic, the system is logically trivial if and only if $\perp$ is inhabited.

As the derivation of a contradiction, that is, the derivation of a formula and its negation, in a formal system with classical logic (intuitionistic logic, and several other logics) as the underlying logic is equivalent to the trivialisation of the system, there is a tendency to confuse inconsistency with triviality and consider that they are just the same logical property. It is the cause of calling (logically) inconsistent the type theories where types corresponding to logic formulae are all inhabited.

As it was explained in Section 1, paraconsistent logics make difference between inconsistent and trivial theories. It is important to clarify that, although it is possible to define inconsistent but not trivial logics, paraconsistent logics are usually not inconsistent logics, but the contradictions arise in the theories that are built over them.

Taking into consideration the differentiation between the concepts of triviality and inconsistency that are made in paraconsistent logics, we shall make analogous differentiations for type theories.

Definition 5.1. Let $\mathbb{F}(\mathcal{L})$ be a set of (logic) formulae generated by some language or signature $\mathcal{L}$. Let T be a type theory and let I be an interpretation of formulae in $\mathbb{F}(\mathcal{L})$ as types of T (that is, I is a function from $\mathbb{F}(\mathcal{L})$ into types of T):

- (i) T is *logically trivial under I* if for every $A \in \mathbb{F}(\mathcal{L})$ there is a term a such that $\vdash_T a : I(A)$ (that is, if all types in T corresponding to formulae in $\mathbb{F}(\mathcal{L})$ are inhabited).
- (ii) T is *logically inconsistent under I* if $\mathcal{L}$ has a negation $\neg$ and there is a formula $A \in \mathbb{F}(\mathcal{L})$ and terms a and b such that $\vdash_T a : I(A)$ and $\vdash_T b : I(\neg A)$.
- (iii) A context Γ of T is *logically trivial under I* if for every $A \in \mathbb{F}(\mathcal{L})$ there is a term a such that $\Gamma \vdash_T a : I(A)$.
- (iv) A context Γ of T is *logically inconsistent under I* if $\mathcal{L}$ has a negation $\neg$ and there is a formula $A \in \mathbb{F}(\mathcal{L})$ and terms a and b such that $\Gamma \vdash_T a : I(A)$ and $\Gamma \vdash_T b : I(\neg A)$.
- (v) T is *paraconsistent under I* if there is a context Γ of T such that Γ is logically inconsistent but not logically trivial.

The type systems λU , λU^- and ITT with **Type : Type** are known to be ‘logically inconsistent’ (see, for instance, [9]). In our terminology, they are logically trivial (and also logically inconsistent). However, as it is pointed out by Hurkens [9], the proof of their ‘logical inconsistency’ do not use *ex falso sequitur quodlibet* at all; so they do not depend on the interpretation of the bottom particle and negation in these type theories. That is, we can remove the bottom particle and negation from the logical language, and under the same interpretation (restricted to formulae without negation), prove that these systems are logically trivial, without being logically inconsistent. If in the logic language there is a negation, of course any logically trivial type theory (or context) would be logically inconsistent.

As PL_5 is a paraconsistent logic (see Theorem 1.4, which also applies to the extension PL_5), by Theorems 4.4.1 and 4.4.6 we have that PTT is paraconsistent, under the propositions-as-types interpretation established in Section 4; it is the reason of why we called this system ‘paraconsistent type theory’. Furthermore, it is possible to show that PL_5 is consistent (using a many-sorted adaptation of the semantics for RFC in [12]), consequently (by Theorems 4.4.1 and 4.4.6 again) PTT is also logically consistent. In the following example, we illustrate the paraconsistency of PTT by showing a particular case of a logically inconsistent but not logically trivial context of PTT , under the propositions-as-types interpretation established in Section 4.

Example 5.2. Let S and $\mathcal{L}_S$ be as given in Example 4.1.2. Then, $P(x_1^A)$ and $Q(x_1^A, x_1^B)$ are atomic formulae in $\mathbb{F}(\mathcal{L}_S)$. Take $\Delta = \{P(x_1^A), \neg P(x_1^A)\}$. By the proof of Theorem 1.4, we have that Δ is an inconsistent but not trivial theory. Particularly, $\Delta \not\vdash_{\text{PL}_5} Q(x_1^A, x_1^B)$. Thus, take the context $[\Delta_{Q(x_1^A, x_1^B)}] = \langle A : U_0, B : U_0, c : A, f : A \rightarrow A, g : A \rightarrow B \rightarrow A, P : A \rightarrow U_0, Q : A \rightarrow B \rightarrow U_0, x_1^A : A, x_1^B : B, x_P : P(x_1^A), x_{\neg P} : \overline{P(x_1^A)}, e_A : A, e_B : B \rangle$. Since $[\Delta_{Q(x_1^A, x_1^B)}] \vdash_{\text{PTT}} x_P : P(x_1^A)$ and $[\Delta_{Q(x_1^A, x_1^B)}] \vdash_{\text{PTT}} x_{\neg P} : \overline{P(x_1^A)}$, then $[\Delta_{Q(x_1^A, x_1^B)}]$ is a logically inconsistent context under the interpretation $[\cdot]$. Moreover, by Theorem 4.4.6, it follows that there is no term a such that $[\Delta_{Q(x_1^A, x_1^B)}] \vdash_{\text{PTT}} a : Q(x_1^A, x_1^B)$, then $[\Delta_{Q(x_1^A, x_1^B)}]$ is not a logically trivial context under the interpretation $[\cdot]$.

6 About non-uniqueness of types in PTT

By uniqueness of types in a type theory T we mean that $\Gamma \vdash_T a : A$ and $\Gamma \vdash_T a : B$ implies that A and B are the same type (usually module a conversion relation on types), for every context Γ , term a and types A and B (see, for instance, [2, Theorem 2.6]). In PTT, as defined in Section 2, there is no uniqueness of types. Consider, the following cases which lead to non-uniqueness of types in PTT:

- (i) If $\Gamma \vdash_{\text{PTT}} a : A$, then $\Gamma \vdash_{\text{PTT}} a : \overline{\overline{A}}$ (by the introduction rule for the opposite of opposite types).
- (ii) If $\Gamma \vdash_{\text{PTT}} a : A$ and $\Gamma \vdash_{\text{PTT}} b : \overline{B}$, then $\Gamma \vdash_{\text{PTT}} (a, b) : A \times \overline{B}$ (by the introduction rule for $\times$) and $\Gamma \vdash_{\text{PTT}} (a, b) : \overline{A \rightarrow B}$ (by the introduction rule for the opposite type of $\rightarrow$).
- (iii) If $\Gamma \vdash_{\text{PTT}} a : \overline{A}$, then $\Gamma \vdash_{\text{PTT}} i(a) : \overline{A} + \overline{B}$ (by the introduction rule for $+$) and $\Gamma \vdash_{\text{PTT}} i(a) : \overline{A \times B}$ (by the introduction rule for the opposite type of $\times$). This holds analogously for $j(a)$.
- (iv) If $\Gamma \vdash_{\text{PTT}} a : \overline{A}$ and $\Gamma \vdash_{\text{PTT}} b : \overline{B}$, then $\Gamma \vdash_{\text{PTT}} (a, b) : \overline{A} \times \overline{B}$ (by the introduction rule for $\times$) and $\Gamma \vdash_{\text{PTT}} (a, b) : \overline{A + B}$ (by the introduction rule for the opposite type of $+$).
- (v) If $\Gamma \vdash_{\text{PTT}} a : A$ and $\Gamma \vdash_{\text{PTT}} b : \overline{B(a)}$, then $\Gamma \vdash_{\text{PTT}} (a, b) : (\Sigma x : A) \overline{B(x)}$ (by the introduction rule for Σ) and $\Gamma \vdash_{\text{PTT}} (a, b) : \overline{(\Pi x : A) B(x)}$ (by the introduction rule for the opposite type of Π).
- (vi) If $\Gamma, x : A \vdash_{\text{PTT}} b(x) : \overline{B(x)}$, then $\Gamma \vdash_{\text{PTT}} (\lambda x) b(x) : (\Pi x : A) \overline{B(x)}$ (by the introduction rule for Π) and $\Gamma \vdash_{\text{PTT}} (\lambda x) b(x) : \overline{(\Sigma x : A) B(x)}$ (by the introduction rule for the opposite type of Σ).
- (vii) If $\Gamma \vdash_{\text{PTT}} a : A$ and $\Gamma \vdash_{\text{PTT}} b : B$, then $\Gamma \vdash_{\text{PTT}} (a, b) : A \times B$ (by the introduction rule for $\times$) and $\Gamma \vdash_{\text{PTT}} (a, b) : (\Sigma x : A) B$ (by the introduction rule for Σ).

Most of the non-uniqueness of types in PTT (items (i) to (vi) of the previous list) are consequences of having defined terms for opposite types as an almost direct translation of the extended BHK-interpretation of what constitutes a prove of $\neg A$, or a refutation of A , via a propositions-as-types interpretation. For instance, as a refutation for $A \vee B$ is a pair (d, e) , where d is a refutation of A and e is a refutation of B , then in the introduction rule for $\overline{A + B}$, the term used is a pair (d, e) , where d is a term of type $\overline{A}$ and e is a term of type $\overline{B}$. Note that pairs are also used in the BHK-interpretation for conjunctions, and consequently for product types. In order to avoid this cause of non-uniqueness of types, we could use t° to indicate that t is a term of an opposite type and add equality rules for terms of the form t° analogous of those for t . For instance, for the opposites of disjoint unions we would have the following introduction and elimination rules instead of the defined in Section 2:

$$\frac{a : \overline{A} \quad b : \overline{B}}{(a, b)^\circ : \overline{A + B}}, \quad \frac{c : \overline{A + B}}{p(c)^\circ : \overline{A}}, \quad \frac{c : \overline{A + B}}{q(c)^\circ : \overline{B}}.$$

And the equality rules would be:

$$\frac{a : A \quad b : B}{p((a, b)^\circ)^\circ = a : A}, \quad \frac{a : A \quad b : B}{q((a, b)^\circ)^\circ = b : B}.$$

Another cause of non-uniqueness of types in PTT (item (vii) of the list above) is that $\times$ is a primitive type constructor in PTT. Moreover, as it was mentioned in Section 2, the type constructor $\times$ cannot be defined by means of Σ in PTT, differently of what happens in intuitionistic type theory. One way to avoid this cause of non-uniqueness is by adding subscripts to pairs in order to indicate if they correspond to a simple or a dependent type: pairs $(a, b)_s$ would be for $\times$ types, while pairs $(a, b)_d$ would be for Σ types. The corresponding projection rules must be defined for each one of the subscripts. Although we are here considering that $\rightarrow$ is also a primitive type constructor, as $\rightarrow$ can be defined by means of Π in PTT (just as in intuitionistic type theory), under this possible definition there is not problem with non-uniqueness of types.

Other causes of non-uniqueness of types are already present in ITT: injections are usually presented without indicating which is the other type in the injection (the one that does not appear in the premises), leading to multiple types for injections, and λ -abstractions without indicating the type of the abstracted variables (it is ‘à la Curry’) also lead to multiplicity of types. The solution for these causes of multiplicity of types are well-known: parametrise injections and add the types of variables in λ -abstractions (that is, use typing ‘à la Church’).

If PTT is modified taking into consideration the previous observations, uniqueness of types could be proven. However, as this property is not necessary here, we prefer to define the terms as we did in Section 2 so as not to unnecessarily complicate the notation.

7 Some observations about equality in PTT

In the 1979 version of intuitionistic type theory, published in [16], Martin-Löf introduces a propositional identity type with a strong elimination rule that makes equality of types be extensional and computationally undecidable. The meaning of equality of types in [16, p. 163] is:

Two canonical types A and B are equal if a canonical object of type A is also a canonical object of type B and, moreover, equal canonical objects of type A are also equal canonical objects of type B , and vice versa. For arbitrary (not necessarily canonical) types A and B , a judgment of the form

$$A = B$$

means that A and B have equal canonical types as values.

As we defined PTT by extending the $\rightarrow, \times, +, \Pi, \Sigma$ intensional fragment of intuitionistic type theory, equality of types in PTT is not extensional. Moreover, in PTT there are types with the same canonical objects that cannot be considered equals. For instance, types $\overline{A \rightarrow B}$ and $\overline{A \times B}$ have the same canonical objects (pairs, as shown in Table 2) and the same equal canonical objects (if the equality rules for these types are considered). Thus, considering the extensional meaning of equality in [16], types $\overline{A \rightarrow B}$ and $\overline{A \times B}$ must be considered extensionally equal. However, types $\overline{A \rightarrow B}$ and $\overline{A \times B}$ cannot be equal because their opposite types have different canonical objects (canonical objects of $\overline{A \rightarrow B}$ are λ -abstractions, while canonical objects of $\overline{A \times B}$ are injections). This shows that equality in PTT cannot be defined in the extensional sense in [16], which is a consequence of our choice of terms for opposite types and it is also related with the fail of substitution by equivalents in PL_5 . In order to allow the definition of an extensional equality in PTT, there are two alternatives:

1. First change the terms used in the rules for opposite types in order to assure uniqueness of types (as described in Section 6), and then define extensional equality in the same way than in [16]. This alternative has the inconvenience that some types that should be considered extensionally equal (like $\overline{A + B}$ and $\overline{A \times B}$), they would not be.
2. Define a kind of ‘strong equality on types’, stating that two types are strongly equal if not only them, but also their opposite types are equal (in the sense of [16]).

8 Recovering consistency for some types

The *logics of formal inconsistency* (LFI’s) is a family of paraconsistent logics introduced by Carnielli, Coniglio and Marcos [3]. In these logics, a connective $\circ$ of *consistency* allows to recover the explosive character of contradictions. The idea is that, in general, from a contradiction not all formulae follows, but only for some ‘consistent formulae’ the contradictions lead to derive any formula. That is, if L is an LFI, it is not provable that $A, \neg A \vdash_L B$, for every formula B , but it is provable that $A, \neg A, \circ A \vdash_L B$, for every formula B . Moreover, in many of the LFI’s it is not provable the consistency of any formula, that is, $\not\vdash_L \circ A$, for any formula A . In other words, the explosion under contradictions (or *ex contradictione sequitur quodlibet*), is not a fundamental logical principle of the LFI’s and the explosive character of contradictions must be established only for some formulae through the theories.

Following the same idea of the LFI’s, if we add the empty type N_0 from intuitionistic type theory to PTT, in order to declare the *consistency* of a particular type A in a context Γ , we must include into Γ a declaration $c_A : A \rightarrow \overline{A} \rightarrow N_0$. In this case, if we have that $\Gamma \vdash_{\text{PTT}} a : A$ and $\Gamma \vdash_{\text{PTT}} b : \overline{A}$, for some a and b , then for every type B will be a term c such that $\Gamma \vdash_{\text{PTT}} c : B$. That is, the context Γ will be logically trivial (under any propositions-as-types interpretation considered).

Concluding remarks and future work

Here, by extending ITT with opposite types, we obtained the paraconsistent type theory PTT. The rules of opposite types in PTT are defined based on the rules for negation in the paraconsistent logic PL_5 . Because of that, it is not a surprise the propositions-as-types correspondence between PL_5 and PTT, but it is necessary to address the technical details at a level that allows us to trust in the validity of such correspondence (it is what we made in Section 4). We also present a translation of PTT into ITT and highlight some properties of PTT as the non-uniqueness of types (which can be avoided by making the changes mentioned in Section 6), the strong normalisation, the difficulties in defining an extensional equality and the way in which the ‘consistency’ of some types can be recovered by the inclusion of an empty type and specific declaration in contexts.

In PTT, we make an important distinction between logically inconsistent and logically trivial contexts, a difference that does not exists in ITT. This difference, and the possibility of having logically inconsistent but not trivial contexts, leaves open the possibility of formalising paraconsistent theories in PTT in a natural way.

The opposite type constructor can be viewed as an operation that transforms types into their ‘duals’. Under a logical (or propositions-as-types) interpretation, the duality is between truth and falsity of propositions (since the type inhabitation of a type $\overline{A}$, interpreted as a proposition, can be understood as A is false). If we interpret types as problems (or tasks), the duality is between

solvability and unsolvability of the respective problems. A more difficult understanding of the duality is when types are interpreted as sets. However, also under the set interpretation, opposite types can be viewed as an operation that establishes a duality between some set operations, although it is difficult to understand in what sense this duality occurs. A better understanding of the duality implicit in the definition of opposite types, the extension of the idea of opposite types to other types (as identity types, well-orderings, finite types, the type of natural numbers and universes) and to other type systems (as the systems in the so-called λ -cube) are questions to be addressed in future work. Since proof assistants based on type theory like AGDA and COQ do not support sharing data constructors between inductive types and they have some syntactical limitations for defining inductive types, the formalisation of the type theory with opposite types on these systems is an interesting and fun future work.

Founding

The first author acknowledges support from Universidad de Antioquia, Medellín, Colombia [grant number 2018-19952] and the second author acknowledges support from Universidad EAFIT, Medellín, Colombia.

References

- [1] Ahmad Almukdad and David Nelson. Constructible Falsity and Inexact Predicates. *The Journal of Symbolic Logic* 49.1 (1984), pp. 231–233. DOI: [10.2307/2274105](https://doi.org/10.2307/2274105).
- [2] Henk Barendregt. Introduction to Generalized Type Systems. *Journal of Functional Programming* 1.2 (1991), pp. 125–154. DOI: [10.1017/S0956796800020025](https://doi.org/10.1017/S0956796800020025).
- [3] Walter Carnielli, Marcelo E Coniglio and João Marcos. Logics of Formal Inconsistency. In: *Handbook of Philosophical Logic*. Ed. by D. Gabbay and F. Guenther. 2nd ed. Vol. 14. Springer, 2007, pp. 15–107. DOI: [10.1007/978-1-4020-6324-4_1](https://doi.org/10.1007/978-1-4020-6324-4_1).
- [4] Thierry Coquand. An analysis of Girard’s paradox. In: *Proceedings, Symposium on Logic in Computer Science: Cambridge, Massachusetts, June 16–18, 1986*. Ed. by Dag Prawitz, Brian Skyrms and Dag Westerståhl. IEEE Computer Society Press, Washington, D.C., 1986, pp. 227–236.
- [5] Hebert B. Enderton. *A Mathematical Introduction to Logic*. 2nd ed. Harcourt/Academic Press, 2001.
- [6] Herman Geuvers and Erik Barendsen. Some Logical and Syntactical Observations Concerning the First-Order Dependent Type System λP . *Math. Struct. in Comp. Science* 9 (1999), pp. 335–359. DOI: [10.1017/S0960129599002856](https://doi.org/10.1017/S0960129599002856).
- [7] G. F. C. Griss. Negationless Intuitionistic Mathematics. In: *Koninklijke Nederlandsche Akademie van Wetenschappen, Proceedings*. Vol. 49. 1946, pp. 1127–1133.
- [8] A. Heyting. *Intuitionism. An Introduction*. 3rd ed. North-Holland Publishing Company, 1971.
- [9] Antonius J. C. Hurkens. A Simplification of Girard’s Paradox. In: *Typed Lambda Calculi and Applications (TLCA 1995)*. Ed. by Mariangiola Dezani-Ciancaglini and Gordon Plotkin. Vol. 902. *Lecture Notes in Computer Science*. Springer, 1995, pp. 266–278. DOI: [10.1007/BFb0014058](https://doi.org/10.1007/BFb0014058).

- [10] Norihiro Kamide. Automating and Computing Paraconsistent Reasoning: Contraction-Free, Resolution and Type Systems. *Reports on Mathematical Logic* 45 (2010), pp. 3–21.
- [11] Norihiro Kamide. Natural Deduction Systems for Nelson’s Paraconsistent Logic and its Neighbors. *Journal of Applied Non-Classical Logics* 15 (2005), pp. 405–435. DOI: [10.3166/jancl.15.405-435](https://doi.org/10.3166/jancl.15.405-435).
- [12] E. G. K. López-Escobar. Refutability and Elementary Number Theory. *Indagationes Mathematicae (Proceedings)* 75.4 (1972), pp. 362–374. DOI: [10.1016/1385-7258\(72\)90053-4](https://doi.org/10.1016/1385-7258(72)90053-4).
- [13] María Manzano. *Extensions of First Order Logic*. Cambridge University Press, 1996.
- [14] Per Martin-Löf. An Intuitionistic Theory of Types. In: *Twenty-five years of constructive type theory. Proceedings of a Congress Held in Venice, October 1995*. Ed. by Giovanni Sambin and Jan M. Smith. Written in 1972 but unpublished. Claredon Press, Oxford, 1998, pp. 127–172.
- [15] Per Martin-Löf. An Intuitionistic Theory of Types: Predicative Part. In: *Logic Colloquium ’73 (Bristol, 1973)*. Ed. by H. E. Rose and J. C. Shepherdson. Vol. 80. *Studies in Logic and the Foundations of Mathematics*. 1975, pp. 73–118.
- [16] Per Martin-Löf. Constructive Mathematics and Computer Programming. In: *Logic, Methodology and Philosophy of Science VI (Hannover 1979)*. Ed. by L. Jonathan Cohen et al. Vol. 104. *Studies in Logic and the Foundations of Mathematics*. 1982, pp. 153–175.
- [17] Per Martin-Löf. Intuitionistic Type Theory. Notes by Giovanni Sambin of a series of lectures given in Padua 1980. Bibliopolis, Napoli, 1984.
- [18] Per Martin-Löf. On the Meanings of the Logical Constants and the Justifications of the Logical Laws. *Nordic Journal of Philosophical Logic* 1.1 (1996), pp. 11–60.
- [19] David Nelson. Constructible Falsity. *The Journal of Symbolic Logic* 14.1 (1949), pp. 16–26. DOI: [10.2307/2268973](https://doi.org/10.2307/2268973).
- [20] Dag Prawitz. *Natural Deduction: A Proof-Theoretical Study*. Dover Publications, 1965.
- [21] M. D. G. Swaen. The Logic of First Order Intuitionistic Type Theory with Weak Sigma-Elimination. *The Journal of Symbolic Logic* 56.2 (1991), pp. 467–483. DOI: [10.2307/2274694](https://doi.org/10.2307/2274694).
- [22] A. S. Troelstra and H. Schwichtenberg. *Basic Proof Theory*. 2nd ed. Vol. 43. *Cambridge Tracts in Theoretical Computer Science*. Cambridge University Press, 2000.
- [23] Dirk van Dalen. Intuitionistic Logic. In: *Handbook of Philosophical Logic*. Ed. by D. M. Gabbay and F. Guenther. 2nd ed. Vol. 5. Springer, 2002, pp. 1–115. DOI: [10.1007/978-94-017-0458-8_1](https://doi.org/10.1007/978-94-017-0458-8_1).
- [24] Heinrich Wansing. Falsification, Natural Deduction and Bi-Intuitionistic Logic. *Journal of Logic and Computation* 26.1 (2016), pp. 425–450. DOI: [10.1093/logcom/ext035](https://doi.org/10.1093/logcom/ext035).
- [25] Heinrich Wansing. Natural Deduction for Bi-Connexive Logic and a Two-Sorted Typed λ -Calculus. *The IfCoLog Journal of Logics and their Applications* 3.3 (2016), pp. 413–440.
- [26] Heinrich Wansing. *The Logic of Information Structures. Lecture Notes in Artificial Intelligence*. Springer-Verlag, 1993. DOI: [10.1007/3-540-56734-8](https://doi.org/10.1007/3-540-56734-8).